

Prague University of Economics and Business
Faculty of Informatics and Statistics



**Large Language Models as a tool for
generating high-level features for text
documents**

BACHELOR THESIS

Study program: Data Analytics

Author: Vojtěch Balek

Supervisor: doc. Ing. Tomáš Kliegr, Ph.D.

Prague, June 2024

Acknowledgements

I want to thank my thesis supervisor doc. Ing. Tomáš Kliegr, Ph.D., for his guidance and support throughout writing this thesis.

My deepest gratitude goes to my parents, who unconditionally supported my studies and made this thesis possible.

Big thanks belong to my friend Kačka, whose mental support and cooperation on projects got me through the three years of studies at FIS.

Abstract

This bachelor thesis investigates the usability of large language models (LLMs) for feature generation from text, evaluating whether LLMs can produce interpretable and usable features for machine-learning tasks. The study uses two labeled datasets: the CORD-19 corpus, consisting of coronavirus research articles with binary labels for high and low citation count, and a dataset of scientific articles from Czech research institutions, with article scores assigned according to the M17+ methodology (ranging from 1 to 5).

Seven categorical features were generated for each dataset using the LLama2 language model. These features were used to train models for binary and ordinal classification tasks. Performance was compared to baseline naive models and models trained on term frequency-inverse document frequency (TF-IDF) and sentence embeddings.

In the CORD-19 dataset, models using LLM-generated features achieved an accuracy of 59.8%, outperforming the baseline dummy classifier (50.2%) but falling short of TF-IDF (62.5%) and sentence embeddings (62.5%). Combining LLM-generated features with article abstract and title texts using the AutoGluon platform achieved the highest accuracy (66.5%), followed by combining TF-IDF terms and LLM-generated features (65.3%).

For the M17+ dataset, the model using LLM-generated features attained an accuracy of 37%, surpassing the naive classifier (18%) and TF-IDF (34.3%). Sentence embeddings achieved the highest accuracy (40.8%), while the AutoGluon model trained on abstract and title text achieved 39.5%.

LLM-generated features enhanced the predictive performance of models and demonstrated higher interpretability compared to traditional bibliometric features. However, a notable limitation is the computational cost; generating features for small datasets (2000-3000 samples) requires tens of hours on high-end hardware.

Keywords

large language models, feature engineering, classification, feature importance, interpretability

Abstrakt

Tato bakalářská práce zkoumá použitelnost velkých jazykových modelů (LLM) pro generování příznaků z textu a hodnotí, zda LLM mohou produkovat interpretovatelné a použitelné příznaky pro úlohy strojového učení. Studie používá dvě označené datové sady: CORD-19 korpus, který se skládá z článků o výzkumu koronavirů s binárními štítky určující vysoký nebo nízký počet citací, a datovou sadu vědeckých článků z českých výzkumných institucí, s hodnocením článků podle metodiky M17+ (v rozmezí od 1 do 5).

Pro každou datovou sadu bylo pomocí jazykového modelu LLama2 generováno sedm kategorických příznaků. Tyto příznaky byly použity k trénování modelů pro binární a ordinální klasifikační úlohy. Výkonnost byla porovnána s naivními modely a modely trénovanými na term frequency-inverse document frequency (TF-IDF) a větných vektorech.

V datové sadě CORD-19 dosáhly modely využívající příznaky generované LLM správnosti 59,8 %, což překonalo naivní model (50,2 %), ale nedosáhlo výkonu TF-IDF (62,5 %) a větných vektorů (62,5 %). Kombinace příznaků generovaných LLM s texty abstraktů a titulů článků pomocí platformy AutoGluon dosáhla nejvyšší správnosti (66,5 %), následovaná kombinací TF-IDF termínů a příznaků generovaných LLM (65,3 %).

Pro datovou sadu M17+ dosáhl model využívající příznaky generované LLM správnosti 37 %, což překonalo naivní model (18 %) a TF-IDF (34,3 %). Větné vektory dosáhly nejvyšší správnosti (40,8 %), zatímco model AutoGluon trénovaný na textech abstraktů a titulů dosáhl 39,5 %.

Příznaky generované LLM zlepšily prediktivní výkon modelů a ukázaly vyšší interpretovatelnost ve srovnání s tradičními bibliometrickými příznaky. Významným omezením je však výpočetní náročnost; generování příznaků pro malé datové sady (2000-3000 vzorků) vyžaduje desítky hodin na špičkovém hardwaru.

Klíčová slova

velké jazykové modely, extrakce rysů, klasifikace, důležitost proměnných, interpretovatelnost

Contents

Introduction	7
1 Large Language Models	8
1.1 Transformer architecture	8
1.2 Quantization	9
1.3 LLama 2	9
2 Machine learning on text data	10
2.1 Representing text data	10
2.2 Evaluation metrics	11
3 Literature Review	16
4 Source datasets	17
4.1 CORD-19 dataset	17
4.2 M17+ research paper evaluation dataset	18
5 LLM Feature engineering	20
5.1 Selected features	20
5.2 Model setup	22
5.3 Feature generation	22
5.4 Replicability of results	23
5.5 Prompt tuning	23
6 Generated datasets	25
6.1 Analysis of result datasets	25
6.2 TF-IDF dataset	39
6.3 SciBERT embeddings	40
6.4 LLM-generated feature dataset	41
6.5 TF-IDF + LLM-Generated features	41
6.6 Overview of datasets	41
7 Classification experiment methodology	43
7.1 Train-test split	43
7.2 Permutation feature importance	43
7.3 Used machine learning algorithms	44
8 Results of classification	51
8.1 Evaluation of LLM-generated features	51
8.2 Dummy classification	53
8.3 TF-IDF classification	54
8.4 SciBERT embeddings performance	56

8.5	Models trained on LLM-generated features	59
8.6	AutoGluon Tabular multimodal model	61
8.7	TF-IDF + LLM features	65
8.8	Result comparison	68
9	Discussion	70
	Conclusion	71
	Bibliography	72
A	Prompts used for feature extraction.	77
B	Feature generation	80
C	Statistical tests of all features	80
D	Train-test split	80

Introduction

In the last two years, large language models (LLMs) have become a mainstream tool for researchers, students, and the general public. The main current use-case of LLMs is natural language generation, which has seen significant progress in recent years and allowed for machine-generated text to be almost indistinguishable from human-generated outputs by untrained evaluators (Clark et al., 2021).

This bachelor thesis aims to evaluate whether large language models, in their current state, can extract features from text that are interpretable and usable in text classification or other tasks, ideally increasing the predictive performance of models trained on these features.

The thesis should complement prior research by Beranová et al. (2023) that focused on predicting the citation count of scientific articles from the CORD-19 corpus (L. L. Wang et al., 2020) with the use of explainable machine learning methods. Furthermore, the thesis aims to complement research by Ngoc (2024) that focused on predicting the human review score of scientific articles in the Czech Republic with machine learning methods. Datasets from both experiments serve as individual inputs for experiments in this thesis.

First, the thesis explains the methods used regarding the large language model application. Then, alternative methods for text representation are explained, along with evaluation metrics used for assessing the classification model performance. Prior research concerning the prediction of citation count and human review score is elaborated upon in Chapter 3. Chapter 4 addresses the datasets of interest and shows how they were preprocessed for further experiments. Chapter 5 provides insight into the feature selection process and experiments regarding obtaining valid outputs from LLMs. Chapter 6 explores the newly created datasets and shows the process of creating various versions of the sets to be used for training in later chapters. Chapter 7 covers the methodology of experiments used to evaluate the predictive performance of generated features. Chapter 8 presents results from classification experiments. Chapter 9 deals with the discussion and limitations of the research presented in this thesis.

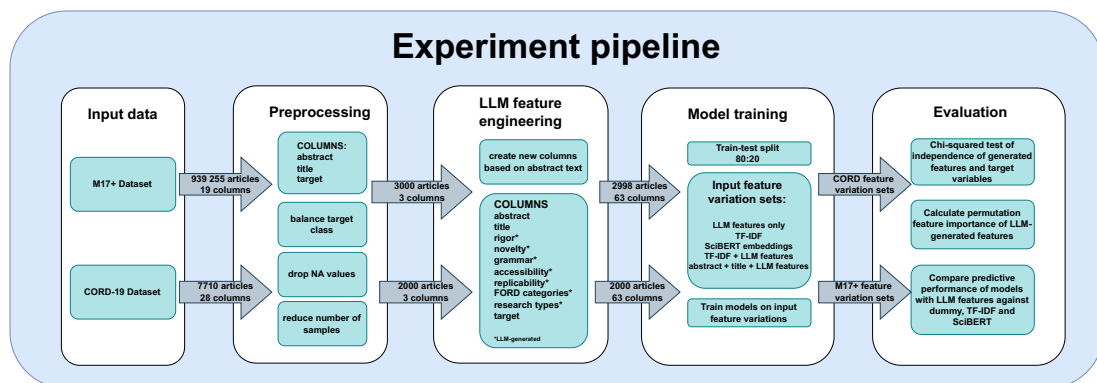


Figure 1: Pipeline of conducted experiments on CORD-19 and M17+ datasets. Source: author

1. Large Language Models

This chapter covers the basic ideas behind large language models (LLMs), model quantization, and finally, the LLaMa2 (Touvron et al., 2023) model used as a model of choice for the main experiment of this thesis.

1.1 Transformer architecture

LLMs are a type of neural network built using the transformer architecture (Vaswani et al., 2023) and mark significant progress in artificial intelligence and its availability to the general public with the release of products such as ChatGPT or Gemini (Anil et al., 2024).

Transformers are based on encoder-decoder architecture. Encoders map input sequence, text, for example, into a sequence of continuous representations, i.e., a scalar vector. The decoder works precisely the opposite - the input is a constant representation, with tokens as an output (Vaswani et al., 2023).

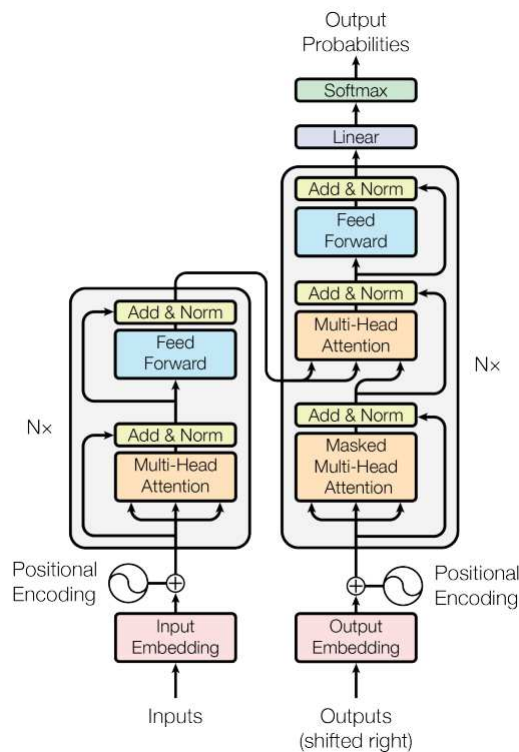


Figure 1.1: Transformer architecture. Source: (Vaswani et al., 2023).

The attention mechanism is critical in transformer architecture, allowing the model to capture dependencies between different tokens in the output sequence. The specific type of attention used in transformers is called self-attention (Vaswani et al., 2023).

1.2 Quantization

Quantization is a technique for decreasing neural networks' computational time and energy consumption. It stores the model weights and activation tensors in a datatype with lower precision than standard 16 or 32-bit representation (Nagel et al., 2021). This typically means storing weights and tensors in 8-bit representation. Neural networks have shown little performance drop after being quantized while decreasing the computational complexity of operations significantly (Nagel et al., 2021).

Due to hardware limitations, a quantized LLama2 model is employed in the experiments described in this thesis.

1.3 LLama 2

LLama2 is a collection of pre-trained open-source large language models developed by Meta (Touvron et al., 2023). The main objective of the release of this model was to provide an open-source model that can compare with closed product models such as ChatGPT based on GPT-3 model (Brown et al., 2020), which are generally heavily fine-tuned and offer state-of-the-art results (Touvron et al., 2023). The model was trained on 2 trillion tokens from publicly available sources, excluding Meta's products (Touvron et al., 2023).

LLama2 offers a variety of models - the model used in this thesis is the LLama2 13B Chat GPTQ. This model has 13 billion parameters and is specifically fine-tuned using reinforcement learning with human feedback for dialogue use cases (Touvron et al., 2023). Quantization is employed as it allows for using a more complex model with more parameters on the same hardware.

There have been several reasons why this model was chosen as opposed to other alternatives. The availability and price of the model played a crucial role. While LLama2's competition, like Gemini from Google (Anil et al., 2024) and ChatGPT, offer API solutions, it would be financially unsustainable as this thesis aims to generate thousands of tokens and make tens of thousands of API calls. Installing and running the model locally on server `xai.vse.cz` maintained by the Department of Information and Knowledge Engineering (KIZI) at FIS VŠE played a crucial role in deciding the model to use.

2. Machine learning on text data

Experiments in this thesis closely relate to binary and ordinal text classifications. Text classification tasks aim to assign correct labels to each instance; in the case of this thesis, those labels are either binary values representing low or high citation count in the case of the CORD-19 dataset or review scores ranging from 1 to 5 in the case of the M17+ dataset. The most common use cases of text classification are sentiment analysis, spam filtering, content moderation, and customer feedback sorting.

This chapter describes text classification's core fundamentals, such as representing text and evaluating classification results.

2.1 Representing text data

Text data is usually transformed into numerical or tabular format to perform various machine learning tasks on text datasets. A variety of algorithms and approaches can achieve this. This section covers two notable text representation techniques: the TF-IDF matrix and the word or sentence vectors (embeddings).

2.1.1 TF-IDF

Term Frequency - Inverse Document Frequency (TF-IDF) (Sparck Jones, 1972) represents text documents based on the relative term frequency while accounting for the term popularity within a corpus. While TF-IDF considers term importance within the document, it does not work with semantic word meanings or differentiate homonyms.

First, the Term Frequency is defined. Suppose we have N documents and let f_{ij} be the frequency (number of occurrences) of a term i in document j . Then we can define the TF_{ij} as follows:

$$TF_{ij} = \frac{f_{ij}}{\max_k f_{kf}}$$

Where $\max_k f_{kf}$ is the maximum frequency of the single term in a document used to normalize the frequency, this means that the most common term in the document has the TF of 1. Suppose that term i appears in n_i out of N documents in the collection. Then, the IDF is defined as:

$$IDF_i = \log_2\left(\frac{N}{n_i}\right)$$

Finally, the TF-IDF score for a term i in document j is calculated as:

$$TF_{ij} \times IDF_i$$

It is a good practice to preprocess the document before calculating TF-IDF scores. The main preprocessing means removing frequently used words with no significant meaning, e.g., articles, conjunctions, prepositions (Rajaraman & Ullman, 2011). Lists of stopwords are generally a part of major NLP-related packages in R and Python and can be sourced from various languages using online sources.

Despite its relatively simple approach, TF-IDF is frequently used in internet library recommender systems (Beel et al., 2016) and for document clustering (Bafna et al., 2016).

2.1.2 Word embeddings

Another way of representing documents is using word or sentence embeddings, also called word vectors (Bengio et al., 2000). Embeddings represent text in a numerical vector and, unlike bag-of-words approaches, can capture the semantic meaning of words or sentences.

In the scope of this thesis, the SciBERT (Beltagy et al., 2019a) model is used, a fine-tuned derivation of BERT, one of the most widely used transformers. SciBERT was trained on a sample of 1,140,000 science papers from Semantic Scholar, utilizing the whole papers, not just the abstracts. The final fine-tuned model significantly outperforms basic BERT in tasks involving scientific text (Beltagy et al., 2019a) and is thus a strong candidate for use in this thesis.

Another notable technique of representing documents as vectors is Word2Vec, which uses a shallow neural network to obtain a high-quality vector representation of text, where semantically close words are positioned closely together in a high-dimensional vector space (Mikolov et al., 2013).

2.2 Evaluation metrics

Evaluation of LLM inference is complex, especially when dealing with an output that is not easily fact-checkable against publicly available sources. One of the most popular but relatively expensive and often having high variance and instability is the human evaluation process (Chang et al., 2024). In the scope of this thesis, it is not feasible to seek to evaluate the correctness of the LLM output itself. While human evaluation would provide a solid case for or against the use of LLMs in the context of high-level feature generation from text, manually evaluating thousands of characteristics of research papers spanning countless scientific fields would be too time-consuming and, given my lack of expertise in most of those fields, unreliable.

Rather than checking the correctness of output itself, the performance of machine learning models trained on the features inferred by the LLama2 model and other traditional methods of representing text such as sentence embeddings (Reimers & Gurevych, 2019) or TF-IDF

(Sparck Jones, 1972) is to be compared.

2.2.1 Confusion matrix

One of the most elementary evaluation techniques is the so-called confusion matrix.

		Actual Value	
		P	N
Predicted value	P	True Positive	False Positive
	N	False Negative	True Negative

Table 2.1: Confusion Matrix for binary classification problem. Source: author

This matrix visually represents the number of successful and unsuccessful classifications in a binary task (Powers, 2008). In multiclass classification, the matrix has K rows and K columns, where K is the number of classes.

True Positives of class k (TP_k) symbolize the number of instances belonging to class k that have been correctly identified as such.

False Positives of class k (FP_k) symbolize the number of instances that have been incorrectly assigned to class k .

False Negatives of class k (FN_k) symbolize the number of instances belonging to class k that have been incorrectly assigned to class other than k .

True Negatives of class k (TN_k) symbolize the number of instances belonging to classes other than k that have been correctly assigned to classes other than k .

2.2.2 Accuracy

Accuracy is an evaluation metric derived from the confusion matrix and represents the ratio of correct classifications to all classifications (Grandini et al., 2020).

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FN + FP}$$

2.2.3 Precision

The precision metric answers the question of what the ratio of correct positive predictions is (Grandini et al., 2020). For the binary classification case, the formula can be presented as follows:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

In the case of multiple classes, the metric is calculated for each class individually:

$$\text{Precision}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FP}_k}$$

When calculating the precision metric for multiclass cases, there are two approaches, each giving a slightly different result.

Macro Precision is an arithmetic mean of precision over all classes. It gives equal weights to all classes and is thus more suitable for balanced sets (Grandini et al., 2020).

$$\text{MacroAveragePrecision} = \frac{\sum_{k=1}^K \text{Precision}_k}{K}$$

Where Precision_k is the precision for class k and K is the total number of classes.

$$\text{MicroAveragePrecision} = \frac{\sum_{k=1}^K \text{TP}_k}{\sum_{k=1}^K \text{Total Column}_k}$$

Total Column_k is number of instances that were predicted as class k .

2.2.4 Recall

Also called Sensitivity or True Positive Rate, Recall represents the ratio of correctly classified actual positives (Grandini et al., 2020). For binary classification it is given by:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

In the case of multiclass classification, the recall is calculated for each class separately like so:

$$\text{Recall}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k}$$

The multiclass recall is divided into two specifications - Micro-recall and Macro-recall. Macro-recall is calculated by taking the average recall for each actual class (Grandini et al., 2020).

$$\text{MacroAverageRecall} = \frac{\sum_{k=1}^K \text{Recall}_k}{K}$$

K is the number of classes, and Recall_k is the recall of class k . As this metric is calculated using the arithmetic mean of recall values of individual classes, it is most useful when the classes are balanced, which is the case in the datasets in this thesis.

Micro-recall is defined as a sum of TP across all classes divided by the sum of all actual positives.(Grandini et al., 2020)

$$\text{MicroAverageRecall} = \frac{\sum_{k=1}^K \text{TP}_k}{\sum_{k=1}^K \text{Total Row}_k}$$

K is the number of classes, and Total Row_k is the number of actual instances of class k .

2.2.5 F1-Score

F1-Score aggregates Precision and Recall and calculates a harmonic mean of them (Grandini et al., 2020). The binary case is defined as follows:

$$\text{F1Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F1-Score calculation is divided into two approaches for multiclass classification, analogically to Precision and Recall metrics (Grandini et al., 2020).

Macro F1-Score is defined as a harmonic mean of MacroAveragePrecision and MacroAverageRecall.

$$\text{MacroF1Score} = 2 \cdot \frac{\text{MacroAveragePrecision} \cdot \text{MacroAverageRecall}}{\text{MacroAveragePrecision} + \text{MacroAverageRecall}}$$

Micro F1-Score gives more weight to larger classes and is thus more suitable for unbalanced sets.

$$\text{MicroF1Score} = 2 \cdot \frac{\text{MicroAveragePrecision} \cdot \text{MicroAverageRecall}}{\text{MicroAveragePrecision} + \text{MicroAverageRecall}}$$

2.2.6 Mean Squared Error

As this thesis also deals with classifying an ordinal variable, it is essential to watch the error of predictions (e.g., misclassifying an excellent science paper as a poor one is a much more severe

error than misclassifying a mediocre one as bad). Due to this, a vital evaluation metric to consider when evaluating ordinal classification tasks is the Mean Square Error (MSE), which punishes significant errors and works quite well even for classification tasks, even though it is mainly used for regression (Gaudette & Japkowicz, 2009). It is, however, less interpretable, as the final error is in units squared.

Mean Squared Error is defined as follows:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

N is the number of samples, y_i is the actual value and $\hat{y}_i$ is the predicted value.

2.2.7 Mean Absolute Error

Mean Absolute Error is a very similar metric to MSE; however, due to the exclusion of squaring the error, the metric doesn't punish extensive violations as much and is preferred when a large number of minor errors is an issue (Gaudette & Japkowicz, 2009).

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

N is the number of samples, y_i is the actual value and $\hat{y}_i$ is the predicted value.

3. Literature Review

This chapter covers the current state of research focused on predicting research article bibliometrics or article scores based on human evaluation. While other authors have done prior research on predicting citations and assessments, none of the sources found dealt with LLM-generated features as inputs for machine-learning methods applied to the scientific text.

One of the critical studies regarding the prediction of citation count was a study by Beranová et al. (2022), which focused on applying explainable machine learning methods on the same dataset used in this thesis, the CORD-19 set with binary target variable representing time-adjusted citation count. The authors of this paper used bibliometric features such as journal Impact Factor (IF), Article Influence Score (AIS), origin author names, number of authors, and other metrics. AIS and IF were the most important features, along with the Anglo-Saxon origin of author names. They also enriched the dataset with named entities from the text. The best BERT-based model achieved 83% accuracy, while other models based on different features ranged from 60% to 72% accuracy.

In Ante (2021), the author delved deeper into studying the impact of abstract readability and its effect on citation count. In this study, the author used logistic regression models and found that a higher readability score, which means more complex text, reduces the likelihood of an article receiving no citations. More complex language requiring prior knowledge in older and larger scientific fields improves the chances of being in the top decile regarding citations.

The critical publication in predicting human evaluation of scientific papers is Thelwall et al. (2023). The authors apply machine learning methods to a dataset of research articles from the United Kingdom submitted to a U.K. Research Excellence Framework in 2021. The authors achieved higher-than-baseline accuracy by enriching the datasets with bibliometrics such as citation counts, number of authors, readability, journal impact score, and abstract text. The best models showed up to 42 % improvement in accuracy over the baseline model.

The thesis by Ngoc (2024), also the source for one of the datasets concerning this thesis, dealt with applying machine learning methods to predict human evaluation scores. The author employed various models and bibliometric features such as article abstracts, keywords, and AIS. The author achieved above-baseline accuracy in his models. However, the research had limitations as the models could not accurately predict all five classes, and the author did not balance the dataset regarding the target variable.

4. Source datasets

This chapter covers the datasets used as inputs for experiments with the large language model. Preprocessing steps and resampling techniques are also covered.

4.1 CORD-19 dataset

The dataset used for generating features of the text is based on the CORD-19 Open Research (L. L. Wang et al., 2020) dataset. The original set consists of nearly 940,000 scientific article metadata sourced from PubMed Central, World Health Organization, PubMed, bioRxiv, medRxiv, and arxiv.org using Semantic Scholar (L. L. Wang et al., 2020) and contains most of the work related to the coronavirus research spanning multiple decades.

The altered CORD-19 dataset (Beranová et al., 2022) was used, which offered a binary label representing high and low citation counts. The binary label was calculated using a multi-step process. First, the number of citations of each article was divided by the age of the publications in years (Belikov & Belikov, 2015; Beranová et al., 2022). After that, the authors created two bins based on the median value of time-adjusted citations. Articles with a time-adjusted citation count lower than the median were labeled 0, while articles with a higher citation count were labeled 1 (Beranová et al., 2022).

As this thesis is focused on utilizing research article abstracts for feature extraction, only the articles with non-missing abstracts were kept. All columns beside abstract, title, and evaluation were dropped.

```
1 from sklearn.utils import resample
2
3 df_low_downsampled = resample(df[df["target"] == 0],
4                               replace=False,
5                               n_samples=1500,
6                               random_state=42)
7
8 df_high_downsampled = resample(df[df["target"] == 1],
9                                replace=False,
10                                n_samples=1500,
11                                random_state=42)
12
13 df = pd.concat([df_low_downsampled, df_high_downsampled])
```

Listing 4.1: Python code used for randomly downsampling categories. Source: author

As shown in figure 4.1, the label categories are unbalanced, as the '0' category makes up approx. 75% of the dataset, while the '1', meaning high citation rate, category is in the minority. Due to the computational limitations of available hardware, the abstract count was reduced from 92,000 to 3,000. The class-balancing downsampling techniques were utilized

to get an equal representation of the '1' and '0' target variables representing high and low citation count, respectively.

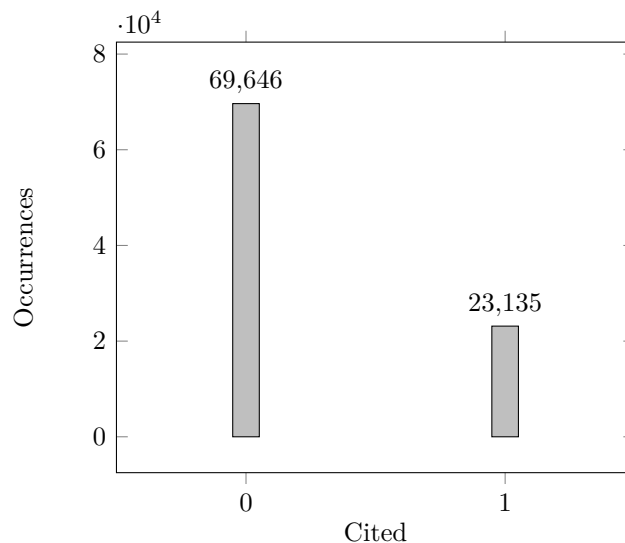


Figure 4.1: Frequencies of target variable in CORD-19 dataset - before downsampling. Source: author

After random subsampling to 1500 occurrences labeled as having a high citation count and 1500 labeled as having a low citation count, it is possible to carry on with feature generation. For this, the `resample()` method from sci-kit-learn library (Pedregosa et al., 2011) with `random_sample = 42` allows for random reproducible downsampling.

4.2 M17+ research paper evaluation dataset

The other dataset used for this thesis is based on articles evaluated by the "Hodnocení výzkumných organizací a hodnocení programů účelové podpory výzkumu, vývoje a inovací dle Metodiky M17+" methodology (Rada pro výzkum, vývoj a inovace, 2017). It comprises 7710 research papers graded 1-5 (1 - best, 5 - worst) by a committee of experts in respective fields according to the M17+ methodology approved by the Czech government in 2017. This dataset has been scraped and maintained during research in Ngoc (2024) and provided to me by the thesis supervisor.

The main reason for adding another dataset to this task is to test my research's replicability and generality. Papers in the source material are not limited to medicinal and encompass various disciplines across all scientific fields. Furthermore, the target variable is not a bibliometric measure but rather a quality metric, which could mean that evaluating this dataset in the text classification task, later on, will provide insight into whether LLM-based features can be used to predict citation counts as well as the supposed quality of the paper based on human evaluation.

When exploring the M17+ dataset, rows with empty abstracts were removed. It was also

decided to rename the original Czech names of the columns to English and name them in the same fashion as in the CORD-19 dataset. Furthermore, all columns besides title, abstract, and evaluation were removed. The class '0' was a class of non-evaluated instances in the target variable. This class was dropped altogether.

As seen in Figure 4.2, the target variable classes are also unevenly distributed, as in the other dataset. It was decided to use the same downsampling technique. As the class '5' in the target variable is heavily underrepresented, the decision was made to go with 400 instances for each class, comprising the final dataset of 2000 instances.

The choice to balance the dataset was made deliberately to ensure the usability of models trained on this dataset in future research. It could be argued that from a use case standpoint, it is to be able to predict all classes. The effect of imbalance was seen in prior research (Ngoc, 2024) and resulted in a model that struggles with correctly classifying classes '1' and '5', potentially failing to warn users of a lack of excellence in their paper if these models were deployed.

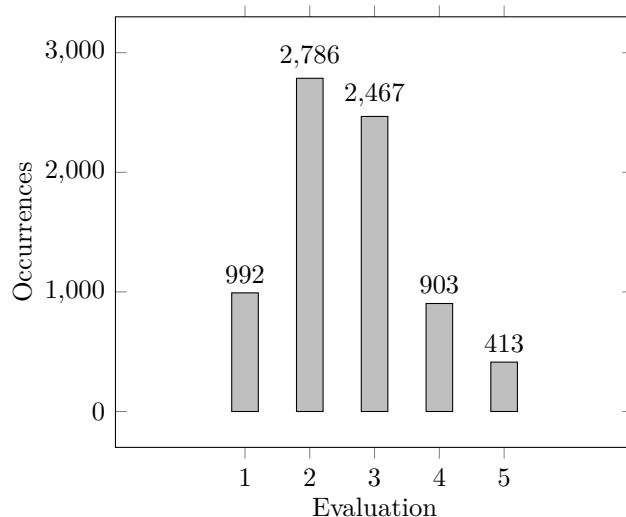


Figure 4.2: Frequencies of target variable before downsampling, M17+. Source: author

After downsampling the dataset to 2000 instances, the dataset is ready to use.

5. LLM Feature engineering

This chapter covers selecting and generating features from article abstracts using LLMs, namely the LLama2 model. Seven features were generated to understand better the nature of articles presented in these datasets. These features include the presence of grammar errors, level of novelty, level of methodological rigor, interdisciplinarity of the research, research types used, replicability of experiments, and accessibility of language in the abstract. This, in total, results in 62 new columns, which is due to interdisciplinarity and research-type features being encoded with dummy encoding.

5.1 Selected features

The selection of features to be generated was based on prior research concerning the features' impact on citation count. All selected features except for **grammar** significantly impacted the number of citations in some cases according to past publications (Annalingam et al., 2014; Ante, 2021; Bruer, 1982; Okamura, 2019; Raff, 2022; Serra-Garcia & Gneezy, 2021; J. Wang et al., 2017). As the M17+ dataset was added as the subject of this thesis after already establishing these features, it was decided to use the same feature set for both datasets.

Both datasets featured in this thesis are composed of research articles. However, both are quite different and concern different fields. So it was impossible to select domain-specific features, such as research types exclusive to the medicinal field, the primary field in the CORD-19 dataset.

A comprehensive look into feature names, variable types, brief descriptions, and possible values is available in Table 5.1.

Feature	Type	Description	Possible Values
research type	categorical nominal	Evaluate the type of research methods used.	Quantitative, Experiment, Exploratory, Qualitative, Mixed methods, Observation, Applied, Descriptive, Focus groups, Survey methodology, Correlational, Interviews, Questionnaires, Secondary, Conclusive, Ethnographic
novelty	categorical ordinal	Evaluate the novelty of the article.	low, medium, high
methodological rigor	categorical ordinal	Evaluate the strength of the logic used in the paper.	low, medium, high
interdisciplinarity	categorical nominal	Estimate which fields the research spans into.	values from Fields Of Research and Development (FORD) (OECD, 2015)
accessibility	categorical ordinal	Estimate the general accessibility of the text based on the language used in the abstract.	low, medium, high
replicability	binary	Based on the abstract, assess if authors mention possible reproducibility of results.	0 - no mention of reproducibility,1 - authors mention reproducibility
grammar	binary	Detect grammatical errors in the abstract.	0 - grammar errors are present, 1 - error-free

Table 5.1: Article characteristics to be evaluated by LLM. Source: author

5.2 Model setup

The parameters used for the model setup were chosen so that the results inferred by the model were as consistent as possible. This specifically required the hyperparameters in `model.generate()` method, which returns the LLM output, to disallow any randomness. This was achieved by setting the hyperparameter `do_sample = False`, which forces the model always to choose the most probable token.

```
1 output = model.generate(inputs=input_ids,
2                           do_sample=False,
3                           max_new_tokens=256)
```

Listing 5.1: Output setup used for inference. Source: author

5.3 Feature generation

The generation itself was done using Python programming language with the use of various libraries; the most notable one here is the Transformers library (Wolf et al., 2019), which offers an all-in-one API for machine learning, NLP, and various other tasks. The model was run locally on a jupyter server of the Department of Knowledge and Information Engineering at FIS VŠE. This server has a dedicated Nvidia RTX A4000 GPU with 16GB of VRAM. This offers sufficient processing power to host the selected Llama2 13B Chat GPTQ model.

```
1 from transformers import AutoModelForCausalLM, AutoTokenizer
2
3 model_id = "TheBloke/Llama-2-13B-chat-GPTQ"
4
5 model = AutoModelForCausalLM.from_pretrained(model_id,
6                                               device_map="auto",
7                                               trust_remote_code=False,
8                                               revision="main")
9 tokenizer = AutoTokenizer.from_pretrained(model_id, use_fast=True)
```

Listing 5.2: Python code for model setup. Source: author

As seen in Listing 5.3, which contains the `generate_features()` method, additional text processing of the LLM output was needed as, in some instances, the model tended to leave artifacts such as HTML tags, whitespaces, or newlines. This method was then applied to every abstract in respective datasets, and based on the input prompt, the final feature was generated.

```
1 def generate_features(abstract, category = None, discipline = None,
2                       prompt_template):
3     input_ids = tokenizer(prompt_template, return_tensors='pt').input_ids.
4     cuda()
5     output = model.generate(inputs=input_ids, do_sample=False, max_new_tokens
6                             =256)
7     features = tokenizer.decode(output[0]).split('Response:')[1]
```

```

5     if discipline:
6         features = features.replace(" ", "").replace("\n", "").replace("</s>",
7         , "")
8         features = features.replace("\n", "")
9         features = features.replace(".", "")
10    else:
11        features = parse_output(features, category)
12    del input_ids
13    del output
14    torch.cuda.empty_cache() # Clear GPU memory
15    return features

```

Listing 5.3: Custom method for model inference. Source: author

As the CORD-19 dataset had 3000 rows, there were $3000 \cdot 62 = 186000$ calls of the LLM. It took the LLM tens of hours of runtime to generate all these features.

The number of inferences for the M17+ dataset with 2000 rows was $2000 \cdot 62 = 124000$ and took tens of hours as well.

Full code used for extracting individual features can be seen in Appendix 9.

5.4 Replicability of results

To demonstrate the reproducibility of my results, it was decided to generate one of the features twice and compare the results to see how the model, given the same conditions, can provide the same results. Feature `novelty` for the CORD-19 dataset was generated twice with the same seed and prompt but two weeks apart. Resulting outputs differed in 9 cases or 0.03% of all instances.

5.5 Prompt tuning

Creating prompts for generating features based on article abstracts was a multi-step process with much trial and error. Each feature had its system prompt specifying the details of the feature and specifics based on which it should be generated.

The final prompts used in the experiment can be seen in Appendix 9.

```

1 prompt_template=f'''[INST] <<SYS>>
2     You are a categorization assistant. Your job will be to assign a certain
3     characteristic to a research paper based on its abstract.
4     In this instance you will assess the rigor of the research.
5     Rigor is a metric of methodological soundness and the strength of logic
6     presented in the abstract.
7     You will choose between three levels of rigor: low, medium and high. Low
8     being the least sound and high being the most.

```

```

6   Be concise, do not provide any explanations. Your answer will consist of
   an answer in json format like so:
7   {{
8       "rigor": "value"
9   }}
10  <</>>
11  Abstract to be evaluated:
12  {abstract}[/INST]
13  Response:'''

```

Listing 5.4: Example of the prompt used for assessing methodological rigor. Source: author

As the M17+ dataset was added as a subject of this thesis after experimenting with CORD-19, the experiments mentioned in the following paragraphs were performed only on CORD-19.

In the first iterations of feature generation, experiments using only one comprehensive prompt to generate all features simultaneously were conducted. This was a time-effective approach, albeit highly inaccurate, to the point of the results being practically unusable. When analyzing the newly generated feature set, it was found out that the model struggled with remembering the context of the prompt and often assigned values that were not in the permitted value sets. This was especially apparent in the 'disciplines' category, where the model was provided with possible values for this feature but failed to consider this context and assigned 128 unique non-permitted values to 1170 rows. This means that 39% of instances had incorrectly classified disciplines.

The subsequent iterations of the prompting process focused on addressing this issue. The first logical step was to divide each category into a separate prompt and focus on providing more context about each feature and its criteria. As for the 'discipline' category, after creating an individual prompt, the model performed even worse than the combined prompt with 317 unique unknown values assigned to 2066 rows, which is approximately 69% of all instances in the CORD-19 dataset.

The final, most accurate approach to respecting the list of permitted values was to ask the model a yes/no question about whether the article belongs to a specific discipline or research category. This produced very few errors and was the final approach utilized in feature generation. As the prompts were more straightforward and LLMs responded with a simple 0/1 evaluation, each inference was relatively fast (5-10 seconds) as opposed to more complicated prompts (20-30 seconds). Still, the total runtime increased dramatically, as each category encompassed in FORD categories and research types was evaluated separately.

6. Generated datasets

This chapter covers the results of experiments with LLM-assisted feature generation. There are two results: the CORD-19 dataset with added features and M17+ with added features. Furthermore, this chapter aims to create different versions of these datasets by generating TF-IDF matrices and SciBERT embeddings for article abstracts. These representations will then serve as a tool for comparison in the upcoming classification task.

6.1 Analysis of result datasets

This section uses fundamental exploratory data analysis to explore the LLM-inferred features for CORD-19 and M17+ datasets. Features generated for both datasets are identical.

When checking the dataset for missing or NA values, no missing or NA values were found; the language model produced some output in all cases for both datasets. After that, instances where the model used values other than the ones disclosed in the prompt were checked. There were two rows in the CORD-19 where the model hallucinated nonsensical output for every feature. As the amount of wrong inputs was very low, they were removed, thus reducing the number of instances in this dataset to 2998. The M17+ dataset had valid values in every column.

The feature-enriched CORD-19 dataset has 2998 rows with 65 columns.

abstract	rigor	novelty	grammar	accessibility	mathematics	comp. science	...	...	...	observation	Cited
Family...	medium	medium	0	high	0	0	...	...	...	0	0
Patients...	medium	low	0	medium	0	0	...	...	...	0	0
Coronavirus...	high	high	0	medium	0	0	...	...	...	0	1
We present...	high	high	1	high	0	1	...	...	...	0	1
Gender...	low	high	0	high	0	0	...	...	...	0	0

Table 6.1: Sample of finalized CORD-19 dataset with added features. Source: author

The feature-enriched M17 dataset has 2000 rows and 65 columns:

abstract	rigor	novelty	grammar	accessibility	mathematics	comp. science	...	...	...	observation	evaluation
This...	high	medium	0	medium	0	0	...	...	...	0	1
Introduction...	medium	medium	1	medium	0	1	...	...	...	0	2
The present...	medium	low	1	medium	0	0	...	...	...	0	4
The main...	low	low	1	low	0	1	...	...	...	0	5
Previous...	medium	low	0	medium	0	0	...	...	...	0	3

Table 6.2: Sample of finalized M17+ dataset with added features. Source: author

The following subsections will explore the distributions of values of features generated by the language model.

6.1.1 Rigor

The **rigor** variable estimates the methodological soundness of logic presented in the abstract. As seen in Table 6.3, more than half of CORD-19 articles were estimated to have high methodological rigor and approximately 7% of low rigor. On the contrary, the articles in the M17+ dataset were almost evenly associated with medium or high rigor, with only 0.95% deemed low in rigor.

	low	medium	high
rigor-cord-19	7.10%	39.86%	53.04%
rigor-m17+	0.95%	49.75%	49.30%

Table 6.3: Relative distributions of **rigor** variable, CORD-19, M17+. Source: author

When plotting the relationship of target variable **cited** and **rigor** for the CORD-19 dataset, we can see that highly rigorous articles are composed chiefly of highly cited articles. On the contrary, articles deemed to show low or medium rigor are mostly less cited. This hints at a possible relationship between these two variables.

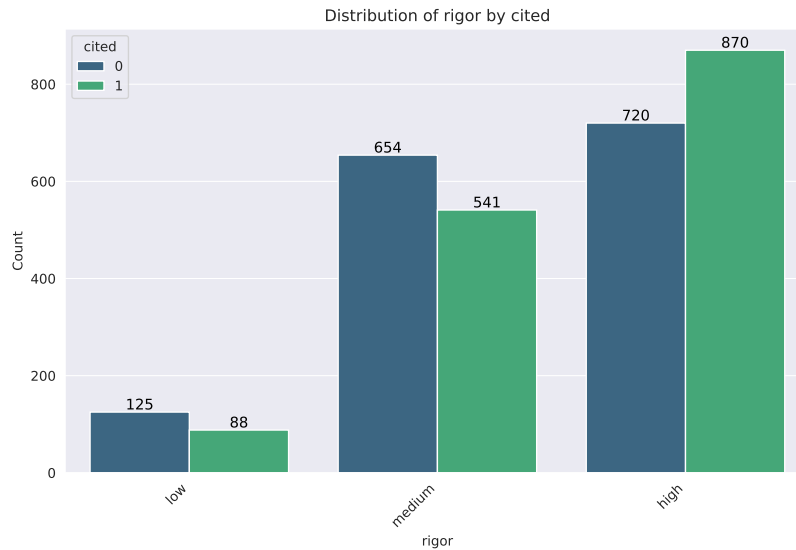


Figure 6.1: Distribution of highly cited and less cited papers by their rigor assessment, CORD-19. Source: author

The same analysis of **evaluation** target variable and **rigor** for the M17+ dataset yields similar results. As shown in Figure 6.2, high rigor is associated with higher-scored articles and medium and low rigor with worse articles. This also hints at a possible relationship between **evaluation** and **rigor**.

The average grade of articles with low rigor is 4.37, with a median of 5. For articles assessed at medium rigor, the mean is 3.55, and the median is 4. Highly rigorous works show a mean of 2.42 and a median of 2.

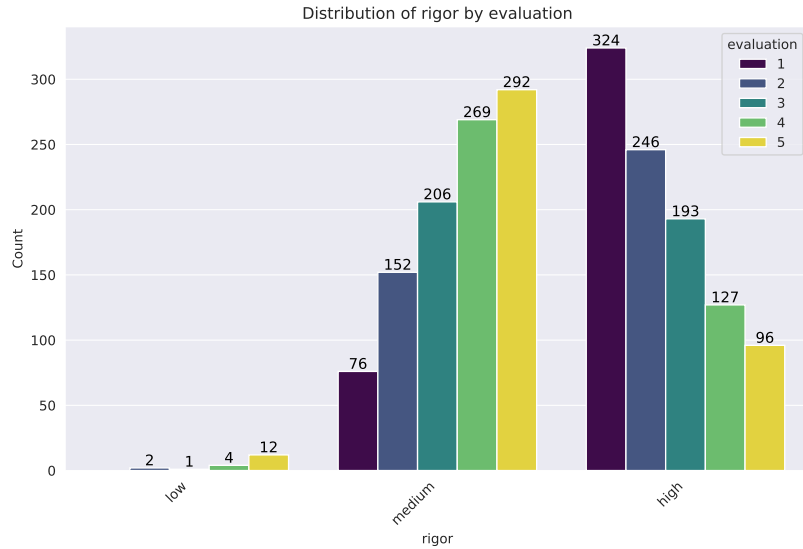


Figure 6.2: Distribution of evaluations of papers by their rigor assessment, M17+. Source: author

6.1.2 Novelty

The **novelty** feature aims to assess the innovativeness of the research presented in the article. Relative distributions of values for **novelty** are shown in Table 6.4. We can see that, generally, CORD-19 articles were assessed to be much more novel than articles in the M17+ dataset. While both datasets have almost the same fraction of articles with low novelty, most of the works in CORD-19 are deemed to be highly novel, while most of the articles in M17+ are estimated to be of medium novelty.

	low	medium	high
novelty-cord-19	20.65%	17.65%	61.70%
novelty-m17+	19.70%	61.70%	18.60%

Table 6.4: Relative distributions of **novelty** variable, CORD-19, M17+. Source: author

Analogically to the **rigor** feature, we can see a pattern in the distribution of **novelty** and **cited**, however less pronounced.

The articles in the M17+ dataset show a distinct pattern - the vast majority of works assessed to be highly novel are evaluated by scores '1' or '2'. In contrast, medium and low novelty articles show fewer highly-scored works.

Articles that scored low in the **novelty** metric have a mean value of the **evaluation** target variable of 3.31 and a median of 3. Articles assessed as medium novelty average a grade of 3.19 and a median of 3. Finally, highly novel articles have a mean grade of 2.03 and a median 2.

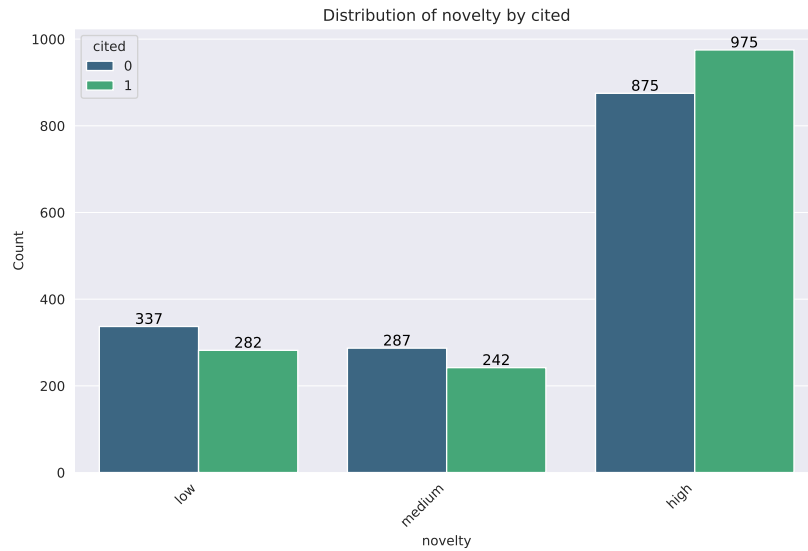


Figure 6.3: Distribution of highly cited and less cited papers by their novelty assessment, CORD-19. Source: author

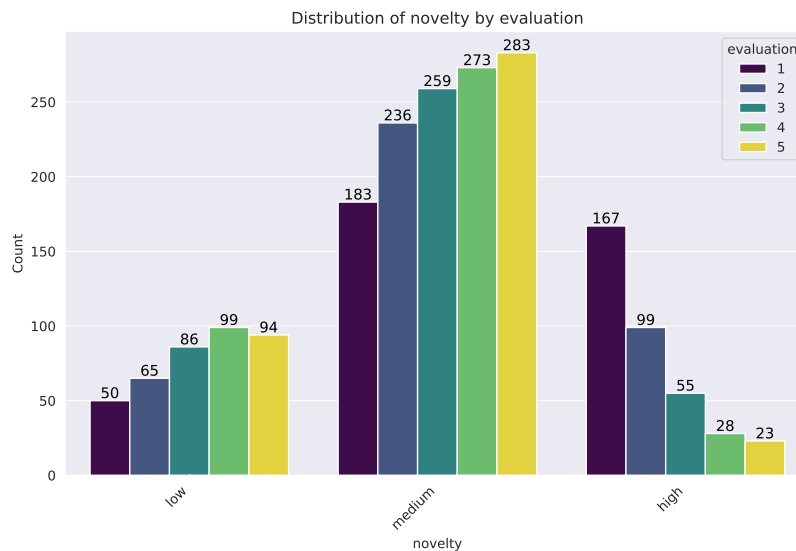


Figure 6.4: Distribution of evaluations of papers by their novelty assessment, M17+. Source: author

6.1.3 Accessibility

The goal of **accessibility** feature is to estimate how easy-to-understand the language used in the article abstract is. Once again, we can see disparities in value distributions between the CORD-19 and M17+ datasets. Most articles in the CORD-19 dataset were estimated to be highly accessible, while most of the articles in M17+ were deemed accessible at a medium level. Poorly-accessible articles were not present in CORD-19 at all and composed only 0.6% of the M17+ articles.

Contrary to previous features, **accessibility** does not seem to show any pronounced pattern

	low	medium	high
accessibility-cord-19	0.00%	8.34%	91.66%
accessibility-m17+	0.60%	93.70%	5.70%

Table 6.5: Relative distributions of **accessibility** variable, CORD-19, M17+. Source: author

in distribution across **cited** or **evaluation** target variables from both datasets. As can be seen in Figures 6.5, 6.6, while slight differences exist in the composition of each categorical bin, these differences are much less pronounced than in previous cases.

Articles assessed to be of low accessibility have an average grade of 3.08 and a median of 3. Similarly, medium accessibility shows a mean value of **evaluation** at 3.02 and a median of 3. Highly accessible articles scored better, with a mean grade of 2.61 and a median of 2.

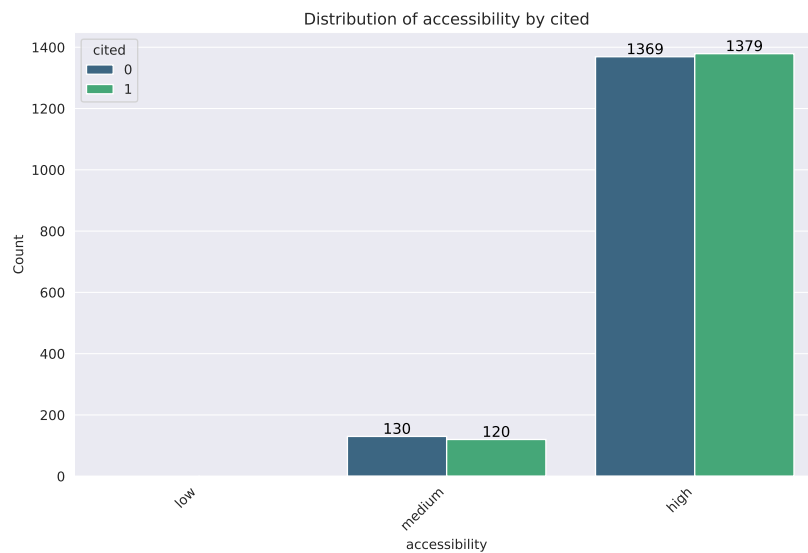


Figure 6.5: Distribution of highly cited and less cited papers by their accessibility assessment, CORD-19. Source: author

CORD-19	0	1
grammar	2754	244
replicability	1142	1856

Table 6.6: Value counts for grammar and replicability features, CORD-19. Source: author

6.1.4 Grammar

Assessing **grammar** aims to distinguish error-free article abstracts from abstracts with grammar errors. In the CORD-19 dataset, over 90% of articles have error-free abstracts, while only about 60% article abstracts from the M17+ dataset are error-free. This is likely due to

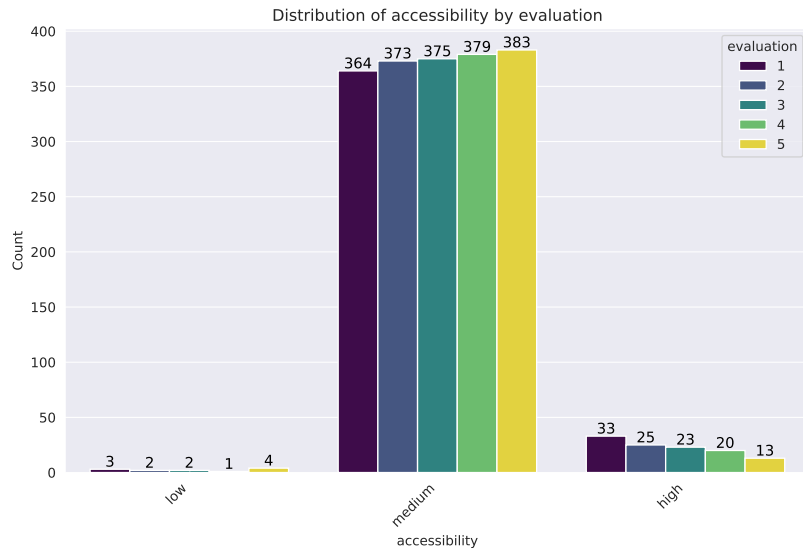


Figure 6.6: Distribution of evaluations of papers by their accessibility assessment, M17+. Source: author

M17+ being a dataset of articles written by authors from Czech universities and institutions whose first language is not English.

	0	1
grammar-cord-19	91.86%	8.14%
grammar-m17+	59.15%	40.85%

Table 6.7: Relative distributions of **grammar** variable, CORD-19, M17+.

In Figure 6.7, we see that cited and not-cited works tend to have an almost identical prevalence of grammar errors in abstracts.

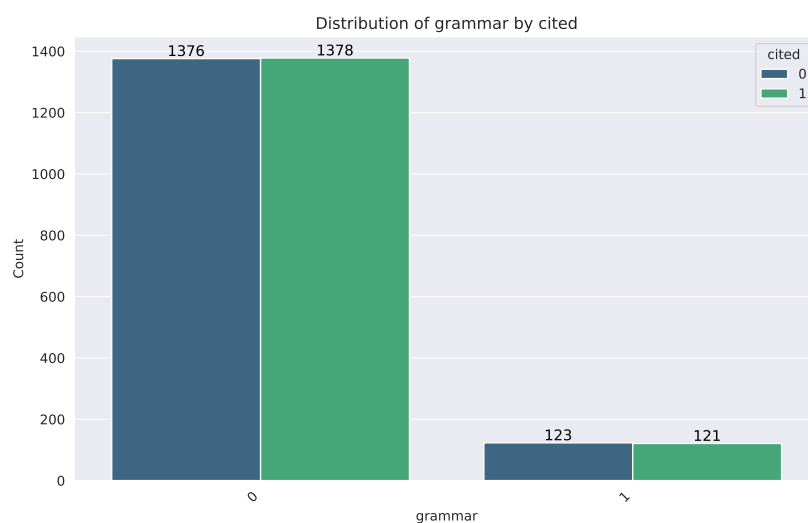


Figure 6.7: Distribution of highly cited and less cited papers by their grammar assessment, CORD-19. (0 - grammar free articles, 1 - articles with grammar errors) Source: author

On the contrary, in the M17+ dataset, articles with error-free abstracts had a more significant proportion of higher-scored articles. In articles evaluated by grade '1', 303 out of 400 are error-free, while only 97 have errors. From the plot, we can assume that the prevalence of grammar errors could indicate article quality or at least play a role in the human evaluation of this quality.

The mean value of `evaluation` for articles with error-free abstracts is 2.74 with a median of 3. Articles with grammar errors in their abstracts have a mean score of 3.37 with a median of 4.

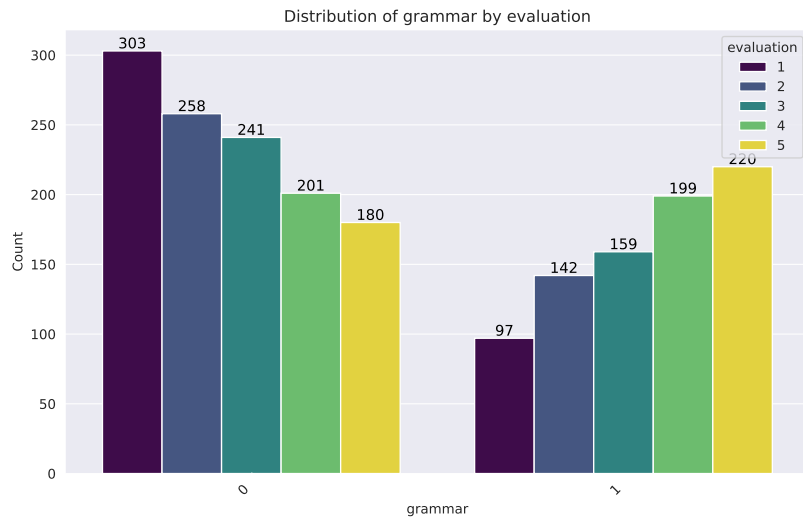


Figure 6.8: Distribution of evaluations of papers by their grammar assessment, M17+. (0 - grammar free articles, 1 - articles with grammar errors) Source: author

6.1.5 Replicability

The `replicability` column was created to identify articles where the authors explicitly mentioned the reproducibility of their research results.

	0	1
replicability-cord-19	38.09%	61.91%
replicability-m17+	58.85%	41.15%

Table 6.8: Relative distributions of `replicability` variable, CORD-19, M17+. (0 - reproducibility of results is not mentioned, 1 - reproducibility of results is explicitly mentioned)

Source: author

Again, in the CORD-19 dataset, differences in distribution can be observed. This pattern is, however, not very pronounced.

In M17+, as seen in Figure 6.10, the pattern is more apparent, with reproducible research generally evaluated at higher grades. Articles that did not mention reproducibility have a mean value of `evaluation` 3.38 and a median of 4. On the other hand, articles that have reproducible research show a mean value of `evaluation` at 2.46 and a median of 2.

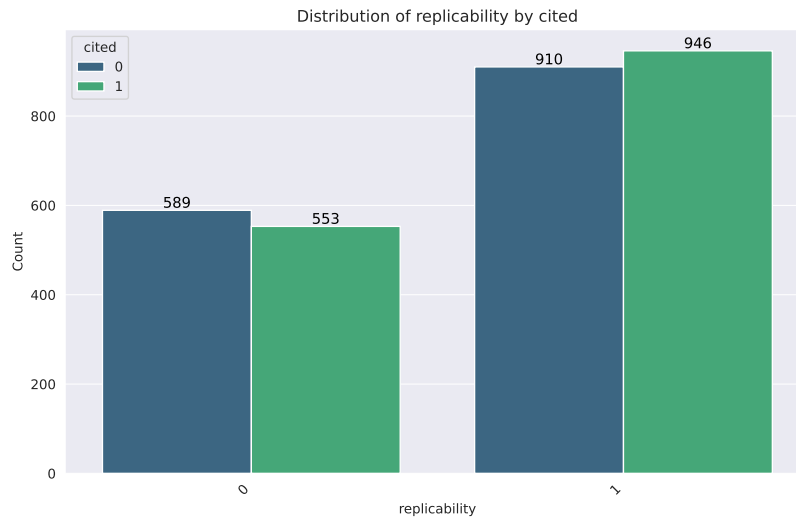


Figure 6.9: Distribution of highly cited and less cited papers by their replicability assessment, CORD-19. (0 - reproducibility of results is not mentioned, 1 - reproducibility of results is explicitly mentioned) Source: author

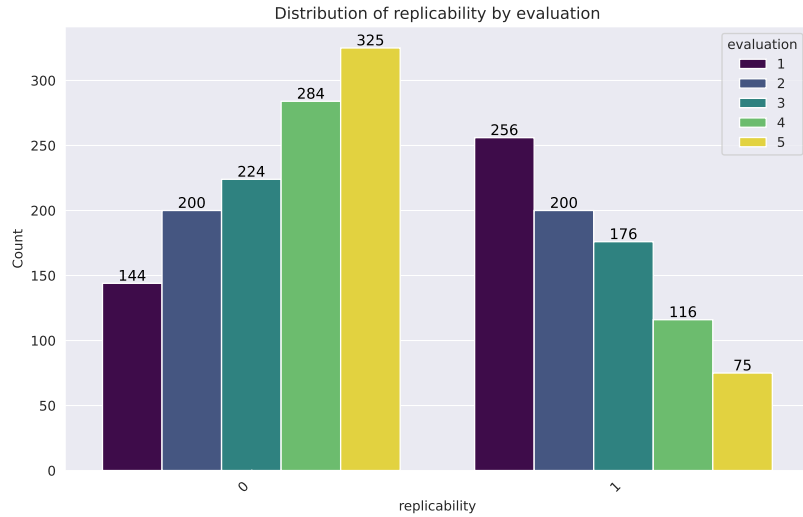


Figure 6.10: Distribution of evaluations of papers by their replicability assessment, M17+. (0 - reproducibility of results is not mentioned, 1 - reproducibility of results is explicitly mentioned) Source: author

6.1.6 Research type

Due to reasons mentioned in Section 5.5, it was necessary to assess research types used throughout the articles with binary yes/no questions. This resulted in `research_type` variable being represented by a sparse matrix of 16 binary variables.

For clarity, instead of thoroughly evaluating all 16 research types, as in the case of previous features, only `applied_research` was presented in this section. Other research types were discussed in a summary in Figures 6.13 and 6.14.

Some articles have not identified as utilizing any research. This can be due to the vague abstract or the LLM's failure to assess the research type. On the other hand, some articles, as per the LLM, have used more than one research method.

In CORD-19, 37.49% of articles seem to employ more than one research method, and 49.73% have not been identified to apply any research method. Approximately 12.78% of articles apply only one research type.

When inspecting the M17+ dataset, it can be seen that only 17.30% of all articles utilize more than one research type, while 72.95% were not identified with any of the provided research types. Only 9.75% of articles showed precisely one method.

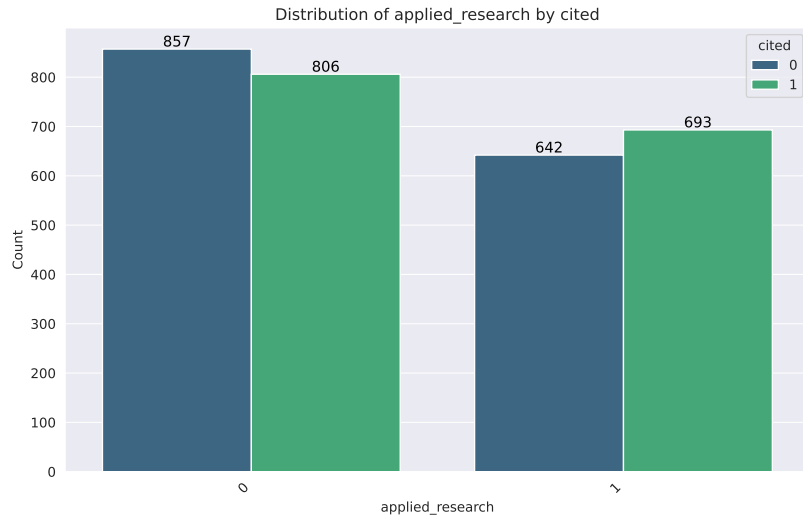


Figure 6.11: Distribution of highly cited and less cited papers by assessment of them containing methods of applied research, CORD-19. Source: author

In Figures 6.11 and 6.12, we can see the distributions of grades across **applied_research** binary variable. For the CORD-19 dataset, we can see that the articles associated with this research type are slightly more cited. In the case of M17+, we can observe that articles that employ applied research as a method tend to have a better evaluation score. The average evaluation of articles not associated with applied study is 3.11, with a median of 3, while the mean for articles that use applied research is 2.56, with a median of 2.

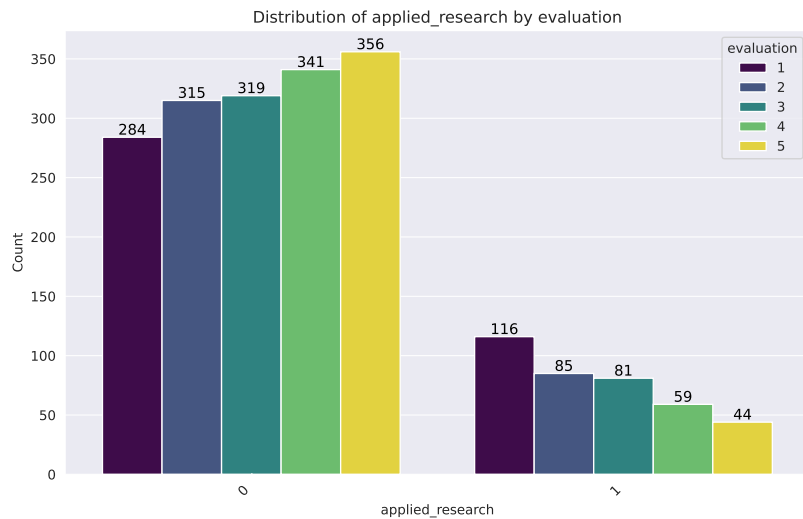


Figure 6.12: Distribution of evaluations of papers by assessment of them containing methods of applied research, M17+. Source: author

Figure 6.13 depicts ratios of highly cited and less cited works. Each bar in the plot represents instances that belong to the given category, i.e., the **applied_research** column represents

all instances where `applied_research` feature equals 1. The red horizontal line represents a 50/50 ratio split between highly cited and less cited works. The last columns `multi_research` and `no_research` represent articles associated with more than one research type and zero research types, respectively. From these plots, it can be observed that there are several research methods associated with significantly lower citation count (ratio of highly cited works lower than 0.4), namely `focus_groups`, `interviews`, and `questionnaires`. No research method shows a significantly high ratio (ratio of highly cited works higher than 0.6) of cited works.

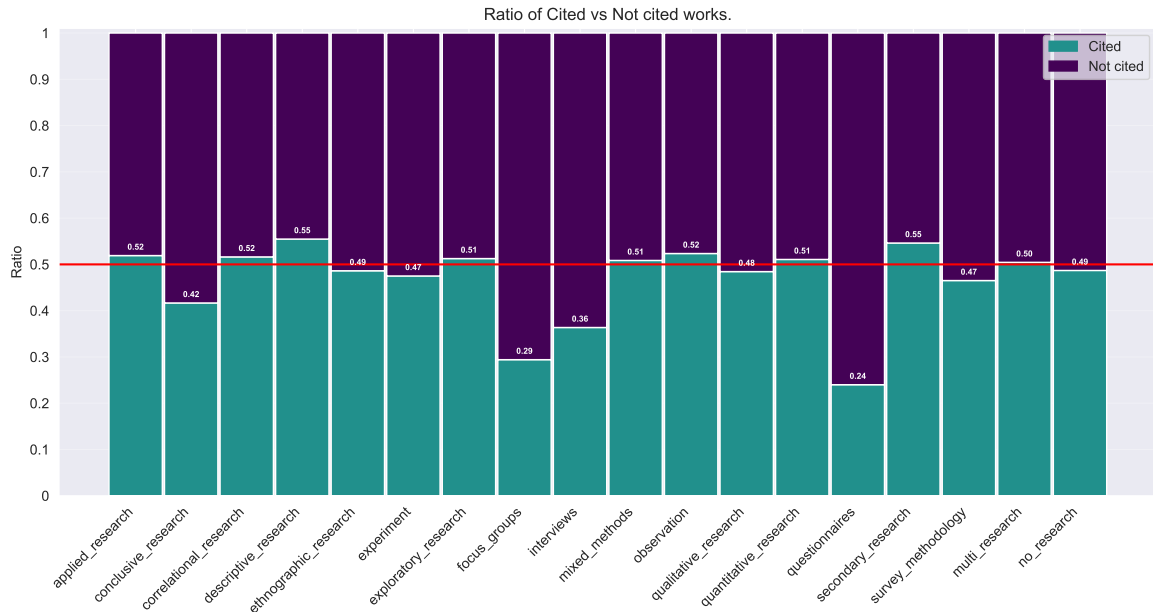


Figure 6.13: Ratio of highly cited and less cited articles assessed by the research present, CORD-19. Source: author

For clarity, in Figure 6.14, the grades '1', '2' and '4', '5' were binned into two groups. This means that we can visualize only 2-dimensional data. The main idea behind this grouping is that set {1,2} should be dissimilar to {4,5} according to the M17+ methodology. Omitting 3-graded works serves as a gap.

Figure 6.14 shows much more variability in the distributions in M17+ than in CORD-19. Several research methods show only a marginal ratio of highly-scored works, namely `conclusive_research`, `focus_groups`, `interview` and `questionnaires`. On the contrary, multiple research methods have also shown a higher prevalence of better-scored works.

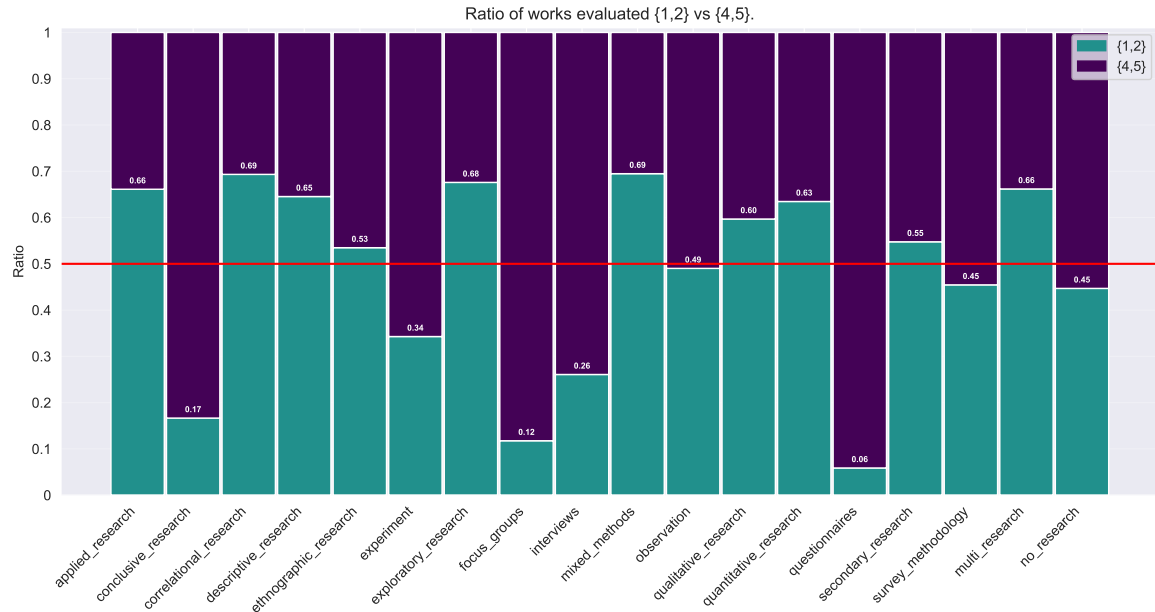


Figure 6.14: Ratio of highly cited and less cited articles assessed by the research present, M17+. Source: author

6.1.7 Interdisciplinarity

Similarly to `research_type`, the `interdisciplinarity` aims to capture all scientific disciplines into which the article delves deep into. This is once again due to the nature of the prompting process represented by a sparse matrix of 41 binary variables where each variable represents a discipline from the Field Of Research and Development (FORD) (OECD, 2015). It was found that 683 instances from the CORD-19 and 399 cases from the M17+ datasets were assigned more than ten disciplines, with some instances being assigned 41 disciplines. This could indicate that some of the LLM outputs are faulty.

Going over all 41 variables individually would be impractical, so once again, for the sake of clarity, only `basic_medicine` was discussed thoroughly, and other FORD categories were shown in summary Figures 6.17 and 6.18.

Out of 2998 articles in the CORD-19 dataset, 55.02% were assessed to delve into more than one discipline, 32.45% were not assigned to any FORD discipline, and 12.53% exactly one.

Out of the M17+ sample, 50.85% of articles that are assumed to concern more than one discipline, 30.80% were not assigned to any scientific field, and 10.35% were assigned to exactly one.

The `basic_medicine` binary variable was selected as a showcase.

In Figure 6.15, a clear distinction in the distribution of highly cited works, indicating a relationship between `basic_medicine` and `cited` can be seen.

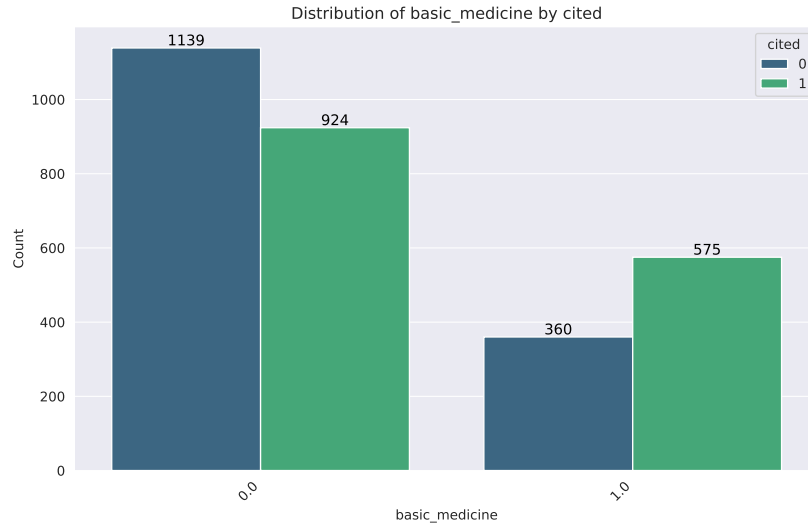


Figure 6.15: Distribution of highly cited and less cited papers **basic_medicine**, CORD-19. (0 - articles not assigned to basic medicine FORD category, 1 - articles assigned to basic medicine FORD category) Source: author

Similarly, in the M17+ dataset, it is apparent that medicinal articles are composed of higher-rated works. The mean evaluation for works not associated with basic medicine is 3.22, with a median of 3. On the other hand, works that fit into the category have an average evaluation of 2.12 and a median of 2.

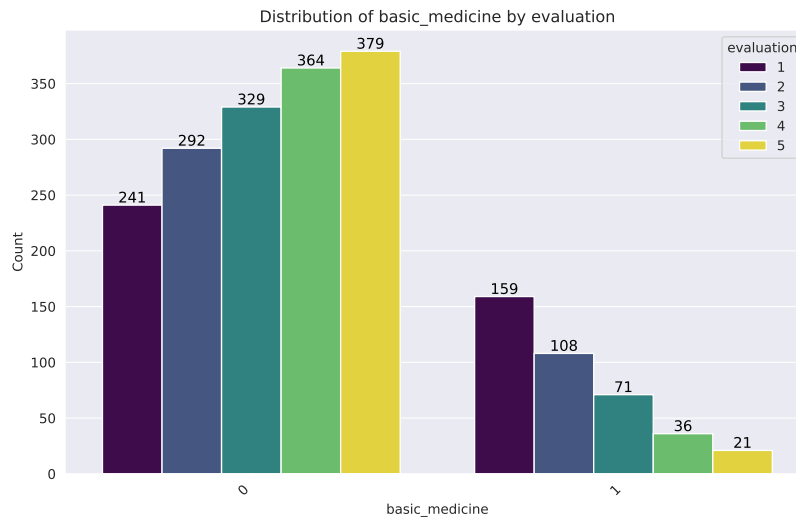


Figure 6.16: Distribution of evaluations of papers by **basic_medicine**, M17+.

In Figure 6.17, we can see the ratio of highly cited and not highly cited articles across disciplines for the CORD-19 dataset. Each column represents all articles that belong to said discipline. Upon close inspection, we can see that being identified as belonging to any category generally means that the article is highly cited. Exceptions are **education**, **art_arts_history**, and **political_science**, which all show a higher ratio of articles with low citation count.

The last columns `multi_disciplinar` and `no_discipline` represent articles that were associated with more than one discipline and zero disciplines, respectively.

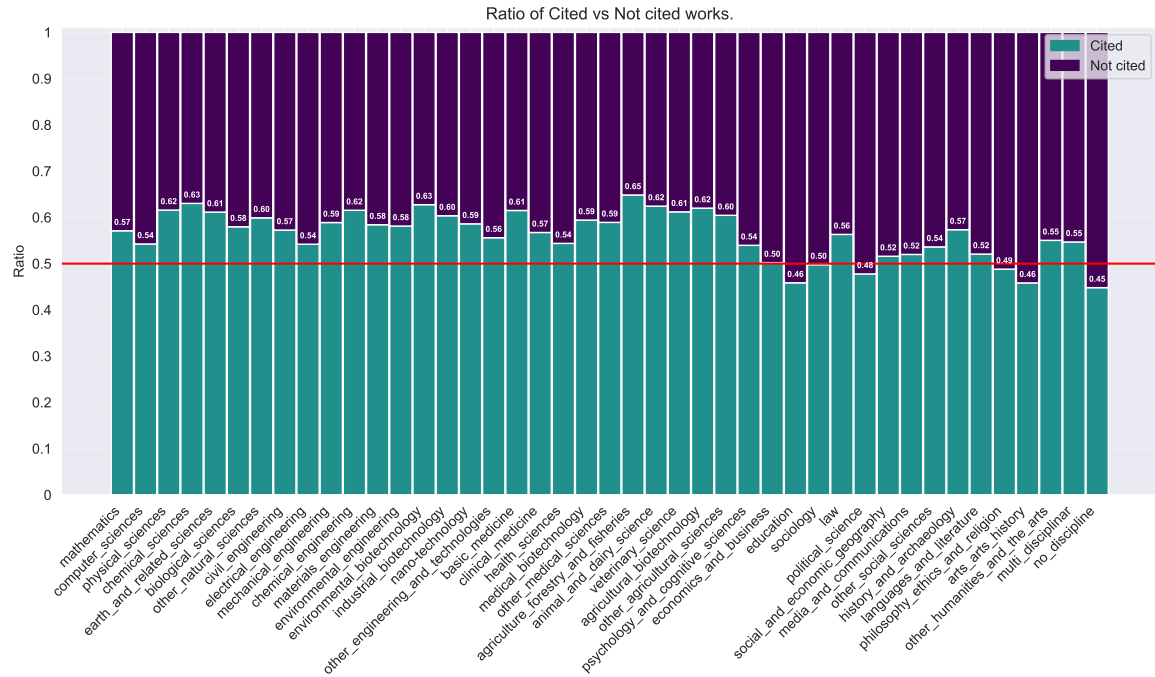


Figure 6.17: Ratio of highly cited and less cited articles assigned to corresponding disciplines, CORD-19. Source: author

As was the case in visualizing research methods, classes '1', '2' and '4', '5' were binned together.

It can be observed that, in general, being associated with any of the disciplines means a higher ratio of good evaluations. Not having any discipline is associated with worse evaluations.

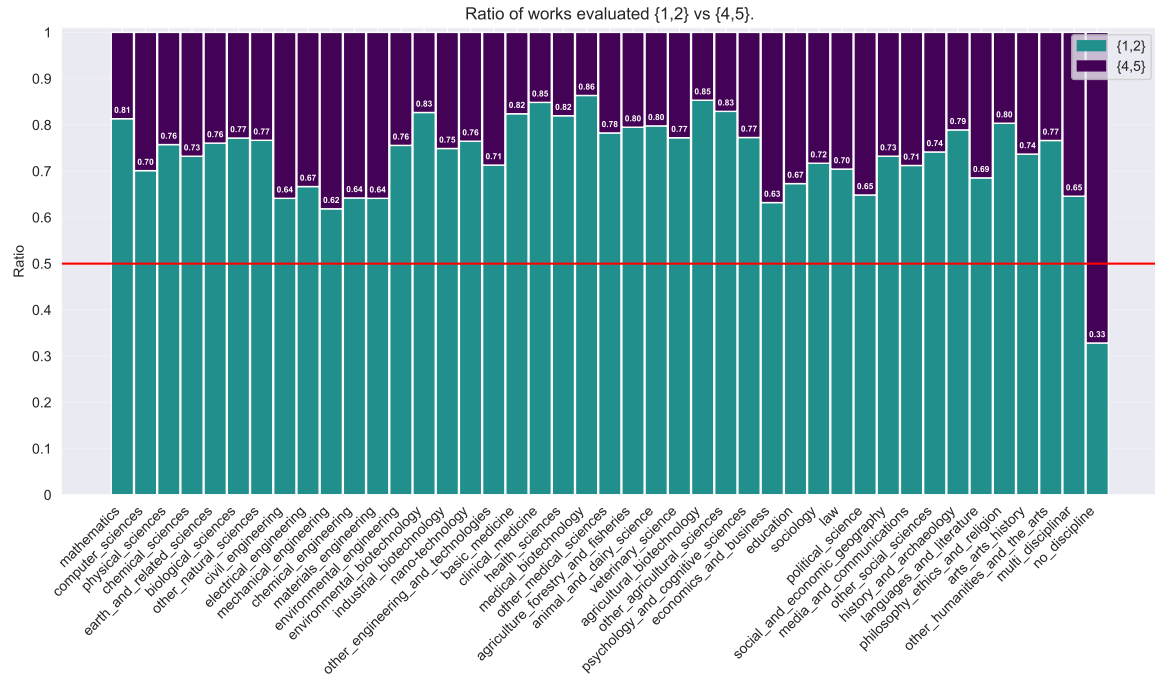


Figure 6.18: Ratio of highly cited and less cited articles assigned to corresponding disciplines, M17+. Source: author

6.2 TF-IDF dataset

Both CORD-19 and M17+ abstracts were preprocessed by removing non-alphanumeric characters, lemmatization, removal of English stopwords, and, finally, tokenization with functions from nltk python library (Bird et al., 2009). These changes were necessary as TF-IDF is an approach that takes words in their current form and could result in one word being represented by multiple forms of itself.

TF-IDF was constructed for unigrams, bigrams, and trigrams. For this experiment, 1500 features were selected using scikit-learn `TfidfVectorizer()` method, as can be seen in Listing 6.2. Method parameters `min_df` and `max_df` were set to 0.005 and 0.80, respectively. These parameters represent a term's minimum and maximum document frequency to be considered in the TF-IDF matrix. The upper bound aims to eliminate domain-specific stopwords, mainly related to the specific language used in research papers. The lower bound of 0.005 signals that terms that appear in less than 0.5 % of the documents should not be considered. This should help eliminate specific words that appear only a handful of times.

TF-IDF matrix is generated after the train-test split. `TfidfVectorizer().fit_transform()` is used on the train set, while only `TfidfVectorizer().transform()` is applied to the test set as that prevents data leakage.

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3 def tf_idf(train, test):
4     vectorizer = TfidfVectorizer(max_features=1500, ngram_range=(1,3), min_df

```

```

=0.005, max_df=0.80)
5 train_tfidf = vectorizer.fit_transform(train['abstract'])
6 test_tfidf = vectorizer.transform(test['abstract'])
7 return pd.DataFrame(train_tfidf.toarray(), columns=vectorizer.
get_feature_names_out()), pd.DataFrame(test_tfidf.toarray(), columns=
vectorizer.get_feature_names_out())
8
9 X_train_c19_tfidf, X_test_c19_tfidf = tfidf(X_train_c19, X_test_c19)
10 X_train_m17_tfidf, X_test_m17_tfidf = tfidf(X_train_m17, X_test_m17)

```

Listing 6.1: Creation of TF-IDF matrices for each dataset. Source: author

The TF-IDF matrix for the CORD-19 dataset has 1500 columns and 2998 rows. The TF-IDF matrix for the M17+ dataset has 1500 columns and 2000 rows.

6.3 SciBERT embeddings

For further performance comparison, the pre-trained SciBERT (Beltagy et al., 2019b) model for embedding the abstracts was used. Each abstract was encoded into a numerical vector of length 768. For this, the Transformers library (Wolf et al., 2019) was used.

```

1 from transformers import AutoTokenizer, AutoModel
2
3 model_name = 'allenai/scibert_scivocab_uncased'
4 tokenizer = AutoTokenizer.from_pretrained(model_name)
5 model = AutoModel.from_pretrained(model_name)
6
7 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
8 model.to(device)
9
10 def get_sentence_embeddings(sentences):
11     inputs = tokenizer(sentences, return_tensors='pt', padding=True,
truncation=True, max_length=512)
12     inputs = {key: value.to(device) for key, value in inputs.items()}
13     with torch.no_grad():
14         outputs = model(**inputs)
15     embeddings = outputs.last_hidden_state[:, 0, :].cpu().numpy()
16     return embeddings
17
18 df_c19['embeddings'] = df_c19['abstract'].apply(lambda x:
get_sentence_embeddings([x])[0])
19 df_m17['embeddings'] = df_m17['abstract'].apply(lambda x:
get_sentence_embeddings([x])[0])

```

Listing 6.2: Creation of SciBERT embeddings from article abstracts for each dataset. Source: author

6.4 LLM-generated feature dataset

Columns `abstract` and `title` were dropped from both datasets. Furthermore, features `rigor`, `novelty`, and `accessibility` needed to be ordinally encoded using scikit-learn’s `OrdinalEncoder` into numerical values.

```
1 enc = OrdinalEncoder(categories=[['low', 'medium', 'high']])
2 df['rigor'] = enc.fit_transform(df[['rigor']])
3 df['novelty'] = enc.fit_transform(df[['novelty']])
4 df['accessibility'] = enc.fit_transform(df[['accessibility']])
```

Listing 6.3: Encoding procedure of ordinal features generated by the LLM.

6.5 TF-IDF + LLM-Generated features

These datasets were created by joining the datasets with LLM-generated features explored in Chapter 6 and TF-IDF matrices based on their abstract from Section 6.2. Both datasets totaled 1562 columns - 1500 terms and 62 LLM-generated features with CORD-19 having 2998 rows and M17+ having 2000 rows.

6.6 Overview of datasets

All final dataset consisting of different features can be seen in Tables 6.9 and 6.10. All train and test sets include the same articles, thus ensuring comparability of results across experiments.

	Number of Features	Number of Columns	Train Size	Test Size
CORD-19 with LLM-generated features	65	2998	2398	600
TF-IDF only	1501	2998	2398	600
SciBERT embeddings only	769	2998	2398	600
LLM-generated features only	63	2998	2398	600
TF-IDF + LLM-generated features	1564	2998	2398	600

Table 6.9: Overview of feature combinations for experiments on the classification task, CORD-19. Source: author

	Number of Features	Number of Columns	Train Size	Test Size
M17+ with LLM-generated features	65	2000	1600	400
TF-IDF only	1501	2000	1600	400
SciBERT embeddings only	769	2000	1600	400
LLM-generated features only	63	2000	1600	400
TF-IDF + LLM-generated features	1564	2000	1600	400

Table 6.10: Overview of feature combinations for experiments on the classification task, M17+. Source: author

7. Classification experiment methodology

This chapter deals with the methodology behind evaluating LLM-generated features by using the LLM outputs as inputs into binary and ordinal classification tasks. The main goal is to test whether or not the LLM-inferred features improve the predictive performance of various models and if they can achieve better-than-baseline accuracy. Thus creating a case for using these features in real-life applications.

Different features derived from the augmented CORD-19 and M17+ will be used to train various models and predict the target variables `cited` and `evaluation`.

7.1 Train-test split

Both augmented datasets CORD-19 and M17+ were split into train and test sets using scikit-learn (Pedregosa et al., 2011) function `train_test_split()`. The ratio of test to train samples was set to 20%: 80%. To ensure the reproducibility of results across all feature combinations, it was necessary to set parameter `random_state = 42` consistently throughout the code.

```
1 from sklearn.model_selection import train_test_split
2
3 def get_train_test(X, y):
4     X_train, X_test, y_train, y_test = train_test_split(X, y,
5                                                         test_size = 0.2,
6                                                         random_state = 42)
7     return X_train, X_test, y_train, y_test
```

Listing 7.1: Method used for train-test split. Source: author

Results of the train-test split for both datasets can be found in Appendix 9.

7.2 Permutation feature importance

To evaluate the importance of features across used models, the permutation feature importance metric of each feature will be calculated. According to this metric, the feature is important if randomly shuffling its values increases the model error (Molnar, 2022). The metric for feature importance in the case of evaluating feature importance in the scope of this thesis is a mean decrease in accuracy.

7.3 Used machine learning algorithms

This section covers the variety of machine learning algorithms that were used with the datasets and showcases their implementations in Python.

7.3.1 Dummy classifier

A dummy classifier is a naive classification that serves as a baseline to beat in most classification tasks.

Hyperparameter `strategy = 'uniform'` ensures that the classification predicts the label randomly, with each label having equal probability.

```
1 dc = DummyClassifier(strategy='uniform', random_state=42)
2 dc.fit(X_train, y_train)
3 y_pred = dc.predict(X_test)
```

Listing 7.2: Dummy classifier. Source: author

7.3.2 Random forest classifier

A random forest is an ensemble learning method (Dietterich et al., 2002) that utilizes multiple decision trees to achieve better classification and regression performance (Breiman, 2001).

For hyperparameter tuning purposes, the grid seen in Listing 7.3 was created.

```
1 grid_rfc = {
2   'max_depth': [10, 20, 30, 40, 45, 50],
3   'max_features': ['log2', 'sqrt', 30],
4   'min_samples_leaf': [30, 40, 50, 60, 70, 90],
5   'min_samples_split': [20, 30, 50, 60, 90],
6   'n_estimators': [100, 200, 400, 1000]}
```

Listing 7.3: Hyperparameter grid space for Random Forest and Gradient Boost. Source: author

This grid was searched over by the `RandomSearchCV()` method. One thousand candidates were selected and fit with the data with 5-fold cross-validation. From these 1000 candidates, the best model was selected.

```
1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.model_selection import RandomizedSearchCV
3 rf = RandomForestClassifier()
4 rf_grid = RandomSearchCV(estimator = rf, param_distributions= grid_rfc,
5   n_iter= 2000, cv = 5, verbose=3, n_jobs = -1, random_state=42)
6 rf_grid.fit(X_train, y_train)
7 best = rf_grid.best_estimator_
```

```
7 y_pred = best.predict(X_test)
```

Listing 7.4: Fit and predictions of the best-performing Random Forest model. Source: author

7.3.3 Gradient boosting classifier

Gradient Boosting (Friedman, 2001) is a machine learning technique that combines the predictions of several weak learners, typically decision trees, to create a strong predictive model. Iteratively optimizing the model's performance using gradient descent effectively minimizes prediction errors and improves accuracy for both regression and classification tasks.

The best estimator is found by a 5-fold cross-validation using `RandomizedSearchCV()` on the same parameter grid in Listing 7.3.

```
1 from sklearn.ensemble import GradientBoostingClassifier
2 from sklearn.model_selection import RandomizedSearchCV
3 gbc = GradientBoostingClassifier()
4 gbc_grid = RandomizedSearchCV(estimator = gbc, param_distributions = grid_rfc
   , n_iter=2000, cv = 5, verbose=3, n_jobs = -1, random_state=42)
5 gbc_grid.fit(X_train, y_train)
6 best = gbc_grid.best_estimator_
7 y_pred = gbc_grid.predict(X_test)
```

Listing 7.5: Fit and predictions of the best-performing Gradient Boosting model. Source: author

7.3.4 AdaBoost classifier

AdaBoost (Freund, Schapire, et al., 1996) is a machine-learning algorithm that combines multiple weak classifiers to form a robust classifier. It adjusts the weights of misclassified instances, improving accuracy for various tasks.

```
1 grid_abc = {
2     'n_estimators': [ 100, 200, 400, 1000],
3     'learning_rate': [0.1, 0.2, 0.4, 1, 1.5]}
```

Listing 7.6: Hyperparameter grid space for AdaBoost. Source: author

I performed 5-fold cross-validation on the parameter grid in Listing 7.6 using scikit-learn method `RandomizedSearchCV()`.

```
1 from sklearn.ensemble import AdaBoostClassifier
2 from sklearn.model_selection import RandomizedSearchCV
3 abc = AdaBoostClassifier()
4 abc_grid = RandomizedSearchCV(estimator = abc, param_distributions = grid_abc
   , n_iter=2000, cv = 5, verbose=3, n_jobs = -1, random_state=42 )
5 abc_grid.fit(X_train, y_train)
6 best = abc_grid.best_estimator_
```

Listing 7.7: Fit and predictions of the best-performing AdaBoost model. Source: author

7.3.5 Support Vector Machines

Support Vector Machines (SVM) is a supervised machine learning technique for classification and regression tasks. They work by finding the optimal hyperplane that separates data into different classes with the maximum margin (Boser et al., 1992) .

After experiments with different kernel functions and other hyperparameters, the best results on both datasets were achieved by the setup mentioned in Listing 7.8.

```
1 from sklearn.svm import SVC
2 svm = SVC(C=1.0, kernel='linear', degree=3, gamma='auto', random_state=42)
3 svm.fit(X_train, y_train)
4 y_pred = svm.predict(X_test)
```

Listing 7.8: Fit and predictions of a Support Vector Machine. Source: author

7.3.6 Logistic regression

Logistic regression is a statistical method for binary or multinomial classification tasks. It models the probability of a binary outcome based on one or more predictor variables using a logistic function (Hosmer Jr et al., 2013).

As logistic regression is not the primary concern of this thesis, the validity of the estimator and checking the usual assumptions was not concluded. Furthermore, it would be more optimal to use the statsmodels (Seabold & Perktold, 2010) library to get better results together with a more detailed regression summary. In the scope of this thesis, using the unified scikit-learn framework was preferred.

Method `LogisticRegressionCV()` is used for a 5-fold cross-validation of results. The parameter `multi_class = 'multinomial'` ensures applicability on the 5-class classification in the case of the M17+ dataset.

```
1 from sklearn.linear_model import LogisticRegressionCV
2 logistic_regression_cv_model = LogisticRegressionCV(
3     multi_class='multinomial',
4     solver='lbfgs',
5     max_iter=10000,
6     cv=5,
7     random_state=42
8 )
9 logistic_regression_cv_model.fit(X_train_pred = logistic_regression_cv_model.
    predict(X_test))
```

Listing 7.9: Fit and predictions of a Logistic Regression model. Source: author

7.3.7 Multilayer perceptron

Multi-layer perception (MLP) is a simple neural network architecture that utilizes more than one fully connected layer of neurons.

For purposes of this thesis, the popular TensorFlow library (Martín Abadi et al., 2015) that provides a framework for deep learning was utilized.

Reproducibility of results is done by `set_random_seed(42)` and `enable_op_determinism()` from the tensorflow library (Martín Abadi et al., 2015).

```
1 import tensorflow as tf
2 tf.keras.utils.set_random_seed(42)
3 tf.config.experimental.enable_op_determinism()
```

Listing 7.10: Setting of seeds that ensure TF and Keras function determinism. Source: author

```
1 from tensorflow.keras.models import Sequential
2 model = Sequential()
3
4 # First Dense layer
5 model.add(Dense(128, activation='relu', input_shape=(X_train.shape[1],)))
6 model.add(BatchNormalization())
7 model.add(Dropout(0.3))
8
9 # Second Dense layer
10 model.add(Dense(64, activation='relu'))
11 model.add(BatchNormalization())
12 model.add(Dropout(0.3))
13
14 # Third Dense layer
15 model.add(Dense(32, activation='relu'))
16 model.add(BatchNormalization())
17 model.add(Dropout(0.3))
18
19 # Output layer for 5-class classification
20 model.add(Dense(5, activation='softmax'))
21
22 model.compile(loss='sparse_categorical_crossentropy', optimizer='adam',
23               metrics=['sparse_categorical_accuracy'])
24 callbacks = [
25     keras.callbacks.EarlyStopping(monitor='val_loss', patience=10, min_delta
26                                   =0.002, restore_best_weights=True),
27 ]
28
29 # Train the model
30 model.fit(X_train, y_train, epochs=30, batch_size=32, validation_split=0.1,
31           callbacks=callbacks)
32
33 # Evaluate the model
34 loss, accuracy = model.evaluate(X_test, y_test)
35
36 # Make predictions
```

```
32 y_pred = np.argmax(model.predict(X_test), axis=1)
```

Listing 7.11: Multilayer perceptron architecture for classification of M17+ articles. Source: author

Selected architecture is a result of trial-and-error experiments. The architecture for deep learning on articles from the M17+ dataset can be seen in Listing 7.11. The model for usage on the CORD-19 dataset is the same but employs a different loss function, different activation function on the final layer, and only one neuron, as the task is binary.

```
1 from tensorflow.keras.models import Sequential
2 model = Sequential()
3
4 # First Dense layer
5 model.add(Dense(128, activation='relu', input_shape=(X_train.shape[1],)))
6 model.add(BatchNormalization())
7 model.add(Dropout(0.3))
8
9 # Second Dense layer
10 model.add(Dense(64, activation='relu'))
11 model.add(BatchNormalization())
12 model.add(Dropout(0.3))
13
14 # Third Dense layer
15 model.add(Dense(32, activation='relu'))
16 model.add(BatchNormalization())
17 model.add(Dropout(0.3))
18
19 # Output layer for binary classification
20 model.add(Dense(1, activation='sigmoid'))
21
22 model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['
    accuracy'])
23
24 # Train the model
25 model.fit(X_train, y_train, epochs=20, batch_size=32, validation_split=0.1)
26
27 # Evaluate the model
28 loss, accuracy = model.evaluate(X_test, y_test)
29 # Make predictions
30 y_pred = (model.predict(X_test) > 0.5).astype(int)
```

Listing 7.12: Multilayer perceptron architecture for classification of CORD-19 articles.

7.3.8 Long Short-Term Memory neural network

Long Short-Term Memory (LSTM) neural networks are a type of recurrent neural network (RNN) that can operate on time-dependent data, such as time series or text, where previous words are important (Hochreiter & Schmidhuber, 1997).

Reproducibility of LSTM outputs is once again ensured by setting random seed functions as seen in Listing 7.10.

In this thesis, LSTM networks are used on the dataset of SciBERT embeddings.

```

1 # Build the expanded LSTM model
2 model = Sequential()
3
4 # First LSTM layer
5 model.add(LSTM(128, return_sequences=True, input_shape=(X_train.shape[1],
6     X_train.shape[2])))
7 model.add(BatchNormalization())
8 model.add(Dropout(0.3))
9
10 # Second LSTM layer
11 model.add(LSTM(64, return_sequences=False))
12 model.add(BatchNormalization())
13 model.add(Dropout(0.3))
14
15 # Dense layers
16 model.add(Dense(64, activation='relu'))
17 model.add(Dropout(0.3))
18 model.add(Dense(32, activation='relu'))
19 model.add(Dropout(0.3))
20 model.add(Dense(1, activation='sigmoid')) # Assuming binary classification
21
22 model.compile(loss='binary_crossentropy', optimizer='adamw', metrics=['
23     accuracy'])
24 model.summary()
25 callbacks = [
26     keras.callbacks.EarlyStopping(monitor='val_loss', patience=5, min_delta
27     =0.002, restore_best_weights=True),
28 ]
29 # Train the model
30 model.fit(X_train, y_train, epochs=20, batch_size=32, validation_split=0.1,
31     callbacks=callbacks)
32
33 # Evaluate the model
34 loss, accuracy = model.evaluate(X_test, y_test)
35 y_pred = (model.predict(X_test) > 0.5).astype("int32")

```

Listing 7.13: LSTM architecture for classification of CORD-19 articles.

```

1 model = Sequential()
2 # First LSTM layer
3 model.add(LSTM(128, return_sequences=True, input_shape=(X_train.shape[1],
4     X_train.shape[2])))
5 model.add(BatchNormalization())
6 model.add(Dropout(0.3))
7
8 # Second LSTM layer
9 model.add(LSTM(64, return_sequences=False))
10 model.add(BatchNormalization())
11 model.add(Dropout(0.3))

```

```

11
12 # Dense layers
13 model.add(Dense(64, activation='relu'))
14 model.add(Dropout(0.3))
15 model.add(Dense(32, activation='relu'))
16 model.add(Dropout(0.3))
17 model.add(Dense(5, activation='softmax'))
18
19 model.compile(loss='sparse_categorical_crossentropy', optimizer='adam',
20               metrics=['accuracy'])
21
22 callbacks = [
23     keras.callbacks.EarlyStopping(monitor='val_loss', patience=3, min_delta
24                                   =0.002, restore_best_weights=True),
25 ]
26 # Train the model
27 model.fit(X_train, y_train, epochs=20, batch_size=32, validation_split=0.1,
28           callbacks=callbacks)
29 # Evaluate the model
30 loss, accuracy = model.evaluate(X_test, y_test)
31 y_pred = np.argmax(model.predict(X_test), axis=1)

```

Listing 7.14: LSTM architecture for classification of M17+ articles. Source: author

7.3.9 Autogluon Tabular

Autogluon is an automated machine-learning platform (Tang et al., 2024). The AutoGluon TabularPredictor() was employed for this thesis. The model can be trained on text inputs and tabular data simultaneously. This allows for a combination of text embeddings, created automatically by the model (Tang et al., 2024), and LLM-based features.

```

1 from autogluon.tabular import TabularPredictor
2
3 predictor = TabularPredictor(label='label')
4 predictor.fit(X_train, hyperparameters = 'multimodal')
5 y_pred = predictor.predict(X_test)

```

Listing 7.15: AutoGluon model used for classification, CORD-19, M17+. Source: author

8. Results of classification

This chapter covers the results of experiments on the LLM-generated features for the CORD-19 and M17+ datasets. First, the statistical significance of select variables regarding the relationship between them and target variables for respective datasets is tested. Then, the LLM-generated features are used as input into a binary and multiclass classification task, and the predictive performance of models trained using these features will be evaluated and compared with models trained on TF-IDF and SciBERT embeddings.

8.1 Evaluation of LLM-generated features

This section will test the formal relationship between LLM-generated features and target variables `cited` and `evaluation`.

For the sake of clarity, the results of a chi-squared test of independence for all LLM-generated features will not be shown, as that would require a table with 62 rows for each dataset. They will, however, be available in an Appendix 9 in the form of a jupyter notebook with outputs.

To achieve this, a chi-squared test of independence (McHugh, 2013) at a 0.05 significance level was used.

8.1.1 Statistical significance tests on CORD-19

The following Null and Alternative hypotheses are formed for the CORD-19 dataset:

Hypothesis 1. H_0 : *There is no significant relationship between being cited and the selected feature.*

Hypothesis 2. H_1 : *There is a significant relationship between being cited and the selected feature.*

Results of hypothesis testing for the CORD-19 dataset show us that out of 5 final features, only two are statistically significant at $\alpha = 0.05$ to assume a sound relationship between them and the number of citations.

feature	d.o.f.	chi-squared value	p-value	significant at $\alpha = 0.05$
Rigor	2	31.26	1.62×10^{-7}	yes
Novelty	2	14.12	8.59×10^{-4}	yes
Grammar	1	0.00	0.95	no
Accessibility	1	0.35	0.55	no
Replicability	1	1.73	0.19	no

Table 8.1: Results of chi-squared test of significance of relation between `cited` and selected generated features, CORD-19. Red marks statistically insignificant features. (d.o.f. - degrees of freedom). Source: author

8.1.2 Statistical significance tests on M17+

As the M17 dataset deals with a completely different target variable representing the papers' quality rather than citation count, a new set of hypotheses is formed.

Hypothesis 3. H_0 : *There is no significant relationship between quality assessment grade and the selected feature.*

Hypothesis 4. H_1 : *There is a significant relationship between quality assessment grade and the selected feature.*

feature	d.o.f.	chi-squared value	p-value	significant at $\alpha = 0.05$
Rigor	2	350.50	7.08×10^{-71}	yes
Novelty	2	240.26	1.99×10^{-47}	yes
Grammar	1	96.81	4.69×10^{-20}	yes
Accessibility	2	12.05	0.15	no
Replicability	1	207.78	7.97×10^{-44}	yes

Table 8.2: Results of chi-squared test of significance of relation between **evaluation** and selected generated features, M17+. Red marks statistically insignificant features. (d.o.f. - degrees of freedom). Source: author

Results of testing the relationship between **evaluation** and generated features indicate that 4 out of 5 showcased features have a statistically significant relationship with the target variable.

8.2 Dummy classification

Naive classification using a uniform dummy classifier yielded expected results that can be used as a baseline for comparison with other models. The dummy classification was done on the test sets of CORD-19 and M17+ with LLM-generated features, abstracts, and titles.

	Accuracy	Recall	F1 Score	MAE	MSE
Uniform classifier	0.502	0.503	0.502	0.499	0.499

Table 8.3: Performance of dummy classifier, CORD-19. Source: author

	Accuracy	Recall	F1 Score	MAE	MSE
Uniform classifier	0.180	0.180	0.180	1.620	4.020

Table 8.4: Performance of dummy classifier, M17+. Source: author

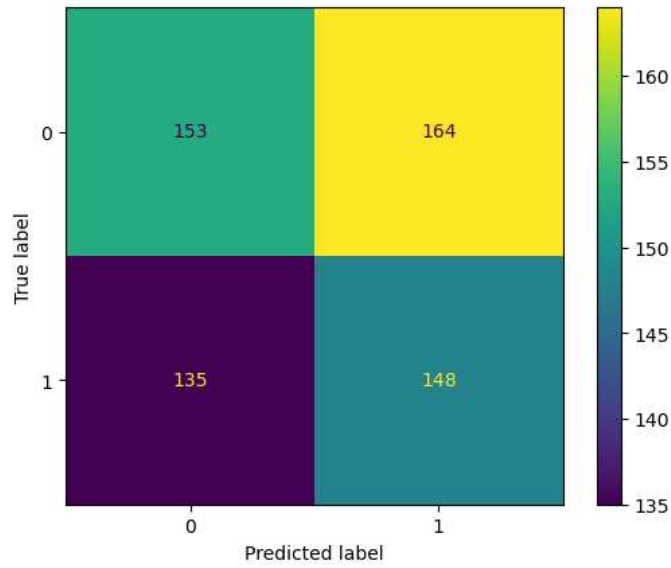


Figure 8.1: Confusion matrix for the dummy model, CORD-19. Source: author

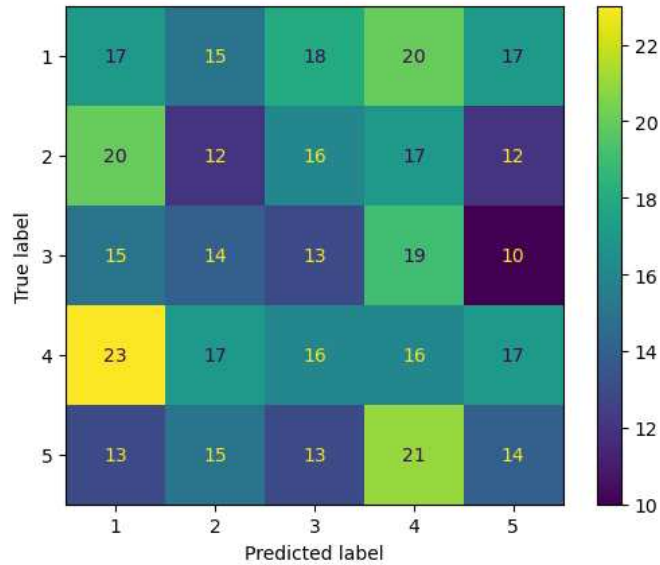


Figure 8.2: Confusion matrix for the dummy model, M17+. Source: author

8.3 TF-IDF classification

This section covers the training of machine-learning models on TF-IDF matrices of CORD-19 and M17+.

The best model for the CORD-19 dataset was AdaBoost. This model achieved 62.5% accuracy, with random forest only being 0.03 percentage points behind. The worst model was logistic regression.

A closer look at the confusion matrix in Figure 8.3 shows that the model, in most cases, predicted that the works are less cited, a total of 323 out of 600 total samples in the test set.

	Accuracy	Recall	F1 Score	MAE	MSE
Random Forest	0.622	0.623	0.621	0.378	0.378
Gradient Boost	0.610	0.610	0.610	0.390	0.390
AdaBoost	0.625	0.622	0.622	0.375	0.375
Logistic Regression	0.574	0.567	0.427	0.427	0.427
SVM	0.617	0.617	0.616	0.383	0.383
MLP	0.597	0.594	0.594	0.403	0.403

Table 8.5: Performance of best models trained on TF-IDF only, CORD-19. Green highlights the best model, and red highlights the worst model by test accuracy. Source: author

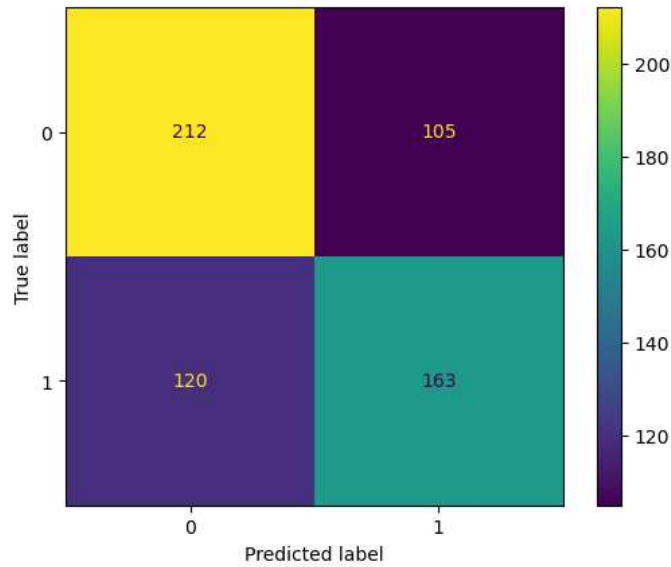


Figure 8.3: Confusion matrix for AdaBoost trained on TF-IDF only, CORD-19. Source: author

In classifying the M17+ dataset, the best-performing model was the Gradient Boost model. The model did not score the best MAE or MSE but was far better in all other metrics, outperforming all other models by a minimum of 2.5 percentage points. AdaBoost achieved the worst performance in all selected metrics.

As seen in the confusion matrix in Figure 8.4, the model makes many misclassifications and struggles with predicting class '3', where the accuracy is close to the accuracy of a dummy predictor. Still, the MAE of 1.073 shows us that the model was, on average, about 1 grade away.

	Accuracy	Recall	F1 Score	MAE	MSE
Random Forest	0.335	0.342	0.301	1.158	2.576
Gradient Boost	0.373	0.369	0.357	1.080	2.345
AdaBoost	0.260	0.260	0.256	1.300	2.880
Logistic Regression	0.330	0.332	0.303	1.090	2.275
SVM	0.343	0.334	0.332	1.073	2.223
MLP	0.348	0.343	0.340	1.155	2.595

Table 8.6: Performance of best models trained on TF-IDF only, M17+. Green highlights the best model, and red highlights the worst model by all metrics. Source: author

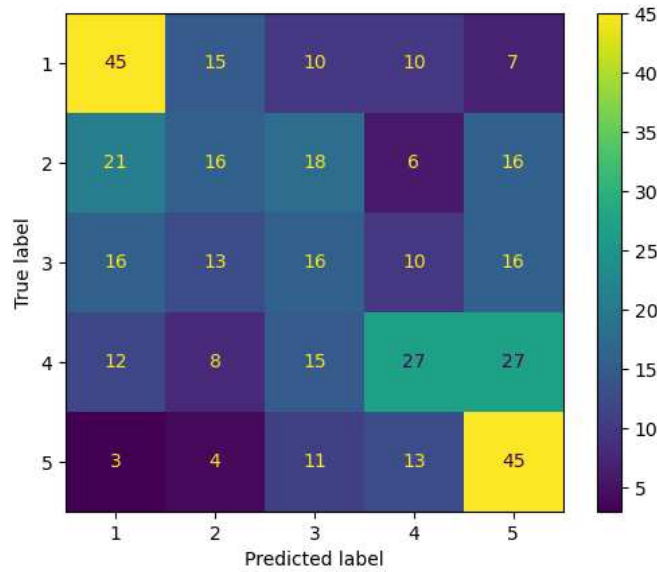


Figure 8.4: Confusion matrix for Gradient Boost model trained on TF-IDF only, M17+. Source: author

8.4 SciBERT embeddings performance

The model of choice trained on SciBERT embeddings of articles from CORD-19 and M17+ datasets was the LSTM neural network.

Regarding model performance on CORD-19 embeddings, the models performed on par with TF-IDF. The best model, multilayer perceptron, is only marginally better than the best model trained on TF-IDF inputs.

In contrast, the performance of models trained on the M17+ dataset SciBERT embeddings was greatly improved across all watched metrics. The confusion matrix of the best LSTM model shows us that the model did very well at identifying classes '1' and '5'. Class '4' was, in most cases, misclassified as '5'. It is worth noting that both models scored MAE lower than 1, which indicates a very good predictive performance.

	Accuracy	Recall	F1 Score	MAE	MSE
LSTM	0.617	0.615	0.615	0.383	0.383
MLP	0.625	0.625	0.625	0.335	0.375

Table 8.7: Performance of best models trained on SciBERT embeddings only, CORD-19. Green highlights the best model, and red highlights the worst model by test accuracy. Source: author

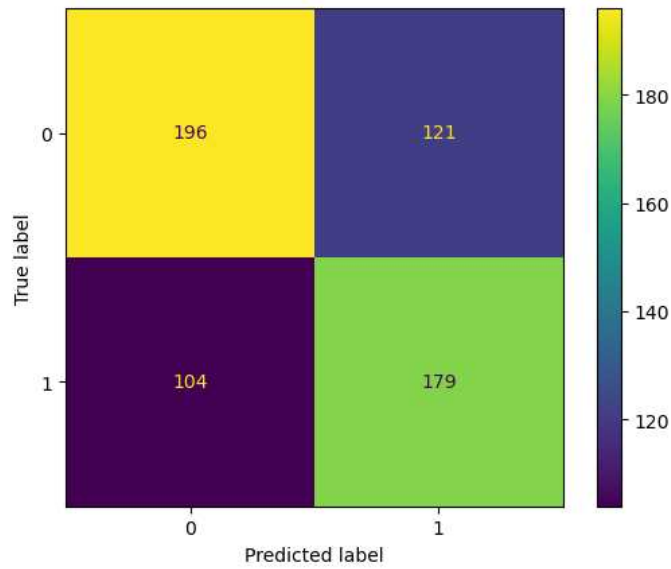


Figure 8.5: Confusion matrix for LSTM trained on SciBERT embeddings only, CORD-19. Source: author

	Accuracy	Recall	F1 Score	MAE	MSE
LSTM	0.408	0.410	0.392	0.848	1.538
MLP	0.395	0.383	0.372	0.925	1.780

Table 8.8: Performance of best models trained on SciBERT embeddings only, M17+. Green highlights the best model, and red highlights the worst model by all metrics. Source: author

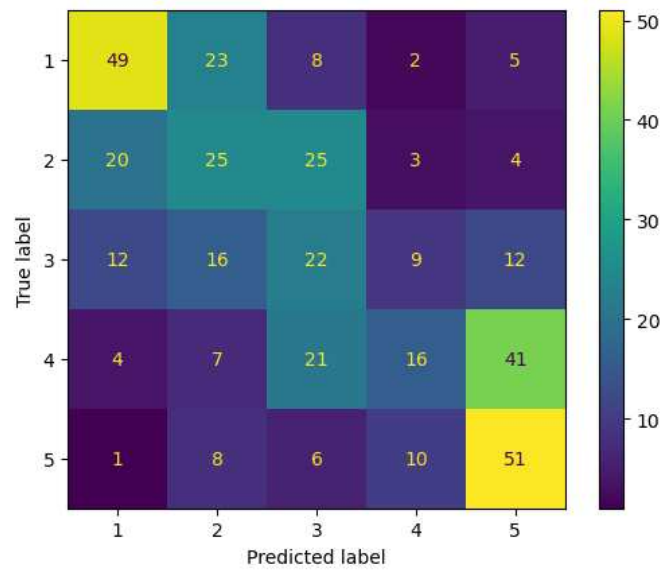


Figure 8.6: Confusion matrix for LSTM trained on SciBERT embeddings only, M17+. Source: author

8.5 Models trained on LLM-generated features

This section contains the specification and evaluation of machine learning methods performed on datasets formed solely from the LLM-generated text features. In Tables 8.9, 8.10, we can see that the performance across models on respective datasets was pretty similar, with the only exceptions being Random Forest and AdaBoost classification trained on M17+, which performed poorly in comparison to other models.

The best-performing models trained on the CORD-19 dataset were accuracy-wise Random Forest and Gradient Boost. For the showcase, the Gradient Boost model was chosen.

	Accuracy	Recall	F1 Score	MAE	MSE
Random Forest	0.598	0.597	0.597	0.402	0.402
Gradient Boost	0.598	0.598	0.598	0.402	0.402
AdaBoost	0.578	0.578	0.577	0.422	0.422
Logistic Regression	0.573	0.567	0.5568	0.427	0.427
SVM	0.563	0.551	0.533	0.437	0.437

Table 8.9: Performance of best models with LLM features only, CORD-19. Green highlights the best model, and red highlights the worst model test accuracy. Source: author

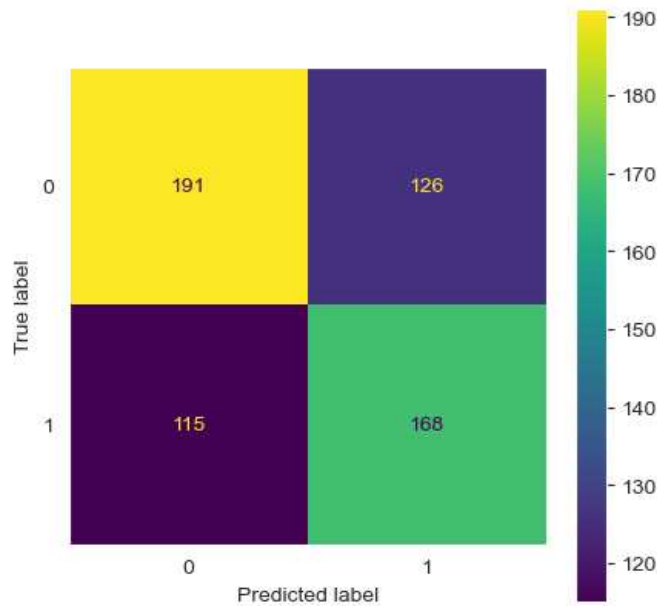


Figure 8.7: Confusion matrix of the Gradient Boost model trained on LLM features only, CORD-19. Source: author

The importance of LLM-generated features when looking at the CORD-19 results shows that belonging to the field of basic medicine and biological sciences were the most important features. Feature **novelty** was also important with an importance of 0.017. After these 3 features, we see a sharp dropoff in the importance of other features.

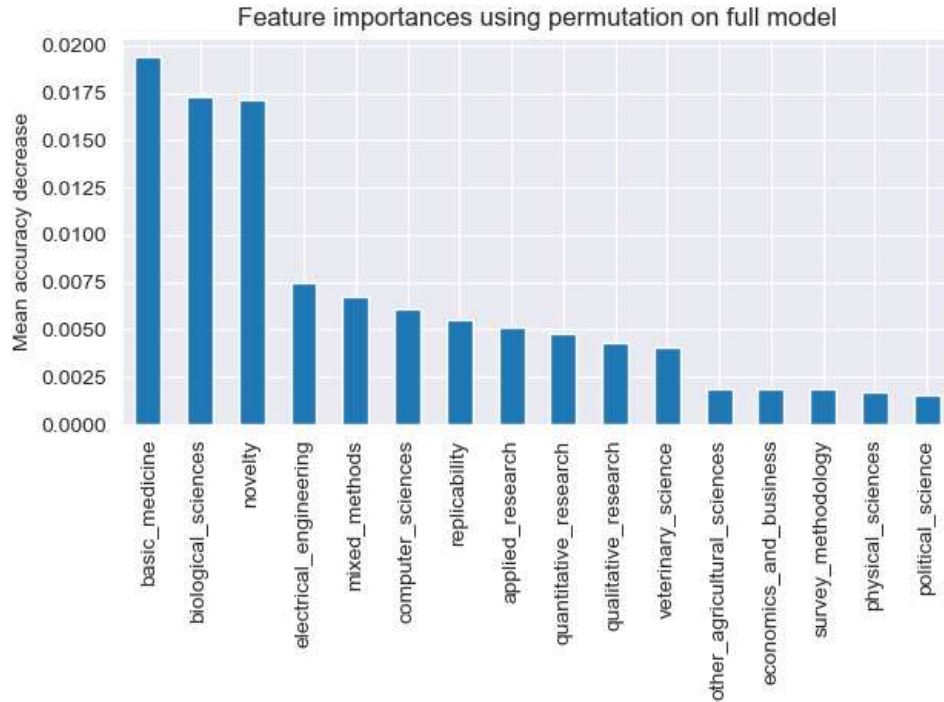


Figure 8.8: Permutation feature importance of the best model with LLM features only, CORD-19. Source: author

When considering all watched metrics, the Gradient Boost performed best. The model performs best when classifying the edge cases of the scale - classes '1' and '5', with the majority being classified correctly as per the confusion matrix in Figure 8.9. The model tends to struggle in other evaluations. Classes '2' and '4' are in most cases misclassified as '1' and '5', respectively. Class '3' is more often misclassified than not.

	Accuracy	Recall	F1 Score	MAE	MSE
Random Forest	0.353	0.350	0.263	1.293	3.258
Gradient Boost	0.370	0.366	0.338	1.103	2.438
Ada Boost	0.340	0.331	0.411	1.093	2.348
Logistic Regression	0.370	0.369	0.324	1.180	2.820
SVM	0.338	0.335	0.307	1.150	2.575

Table 8.10: Performance of best models trained on LLM features only, M17+. Green highlights the best model, and red highlights the worst model by classification metrics. Source: author

Feature importance plot in Figure 8.10 shows that **rigor**, **biological_sciences**, **grammar**, **replicability** and **novelty** were the 5 most important features. Rigor bears a value of 0.45, which can be interpreted as, on average, if the values of **rigor** were randomly shuffled, the model accuracy would, on average, decrease by 0.045. Feature **biological_sciences** has the importance of 0.0273.

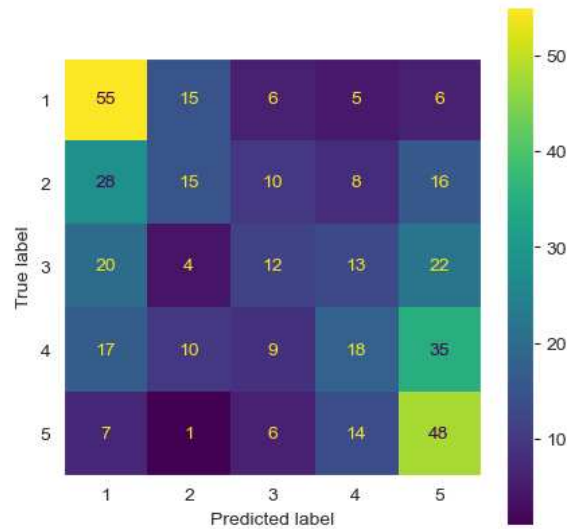


Figure 8.9: Confusion matrix of the Gradient Boost model trained on LLM features only, M17+. Source: author

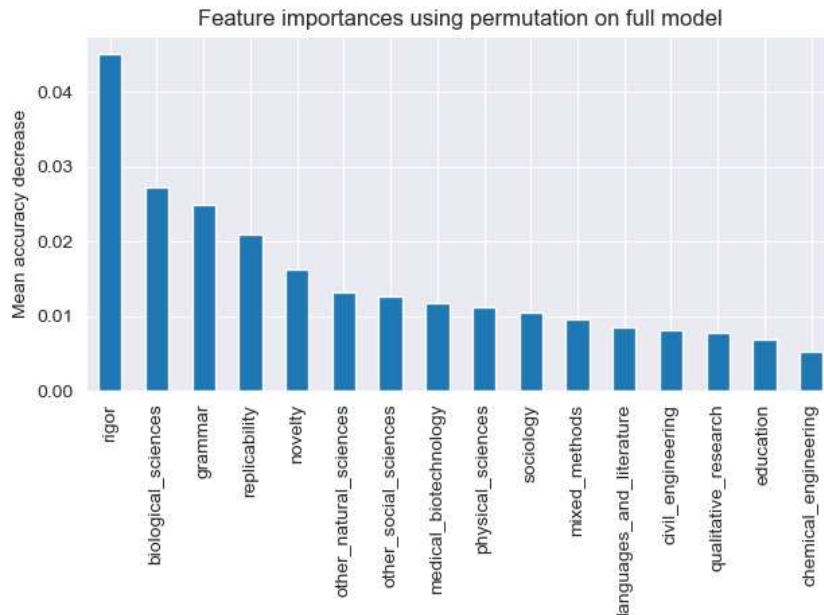


Figure 8.10: Permutation feature importance of the Gradient Boost model trained on LLM features only, M17+. Source: model

8.6 AutoGluon Tabular multimodal model

A total of 3 models were trained on each dataset. AutoGluon’s `TabularPredictor()` as a classifier and predictor on all features available (abstract + title + LLM-generated features). And classifier on text only. This serves as a direct comparison in the change in predictive performance when the LLM-generated features are added.

From models trained on the CORD-19 dataset, the AutoGluon TabularPredictor Classifier was the best and performed better than any other model I have tried so far, achieving 0.665

	Accuracy	Recall	F1 Score	MAE	MSE
Classifier	0.665	0.667	0.664	0.335	0.335
Regressor	0.645	0.651	0.643	0.355	0.355
Classifier text only	0.630	0.636	0.628	0.370	0.370

Table 8.11: Performance of AutoGluon Tabular trained abstract + title + LLM features, CORD-19. Source: author

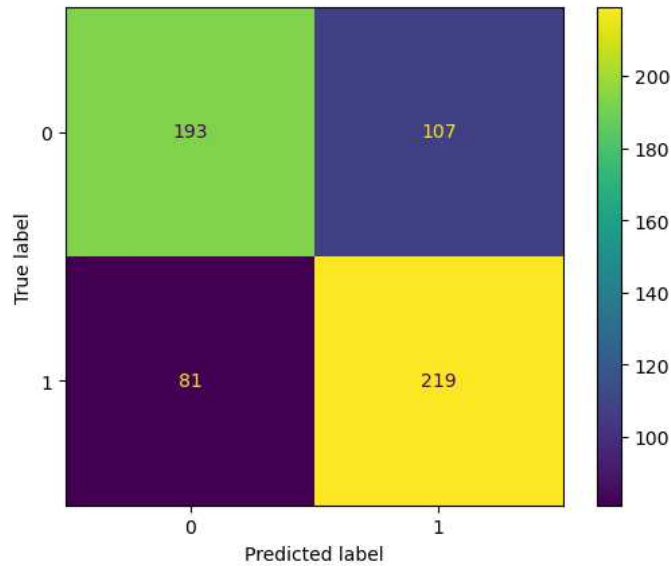


Figure 8.11: Confusion matrix of of AutoGluon Tabular classifier trained on abstract + title + LLM features, CORD-19. Green highlights the best model, and red highlights the worst model test accuracy. Source: author

accuracy. The model performed better when predicting class '1', which corresponds to highly cited works. The control model trained on text only performed worse than both of the datasets enriched by LLM features in all metrics.

As can be seen in Table 8.12, the most important feature for model train on CORD-19 set was **abstract** with permutation feature importance 0.105, followed by **biological_sciences** with 0.028.

Results for the M17+ dataset show us that while the classification with text only outperformed the regressor with text and LLM features in terms of accuracy, it scored worse in every other metric, producing an MAE greater than 1 and an MSE greater than 2. The classifier trained on abstract, title, and LLM features performed best and reached 0.395 accuracy, with an MAE of 0.855. The confusion matrix shows that the model is capable of predicting classes '1' and '5' in most cases while significantly improving the predictions in '2', '4' compared to naive predictor. Class '3' is once again hard to predict.

The permutation feature importances for the AutoGluon model trained on the M17+ dataset in Table 8.10 suggest that the abstract was the key feature, while LLM-generated charac-

Feature	Importance	Std. dev.	p-value
abstract	0.1053	0.0064	0.0000
biological_sciences	0.0290	0.0045	0.0001
title	0.0283	0.0101	0.0016
basic_medicine	0.0257	0.0053	0.0002
rigor	0.0150	0.0041	0.0006
chemical_sciences	0.0110	0.0038	0.0015
descriptive_research	0.0090	0.0019	0.0002
education	0.0047	0.0070	0.1055
languages_and_literature	0.0047	0.0040	0.0293
sociology	0.0033	0.0012	0.0016
computer_sciences	0.0033	0.0020	0.0109
qualitative_research	0.0030	0.0007	0.0004
industrial_biotechnology	0.0030	0.0027	0.0352
applied_research	0.0020	0.0007	0.0019
observation	0.0020	0.0007	0.0019
mixed_methods	0.0017	0.0012	0.0171
correlational_research	0.0017	0.0012	0.0171
other_natural_sciences	0.0013	0.0007	0.0081

Table 8.12: Permutation feature importance of AutoGluon Tabular classifier, CORD-19. Only statistically important features are shown. Source: author

	Accuracy	Recall	F1 Score	MAE	MSE
Classifier	0.395	0.393	0.389	0.855	1.465
Regressor	0.340	0.400	0.330	0.835	1.215
Classifier text only	0.377	0.369	0.324	1.001	2.100

Table 8.13: Performance of AutoGluon Tabular trained abstract + title + LLM features, M17+. Green highlights the best model, and red highlights the worst model by classification metrics. Source: author

teristics played only a minor role. Out of 64 features, 13 were statistically significant. The most important LLM-generated features were **animal_and_dairy_science**, **grammar**, and **basic_medicine**. **Novelty** and **rigor** were also important. Other features concerned mostly fields of study; the only relevant feature that concerned the research field was the feature **correlational_research**.

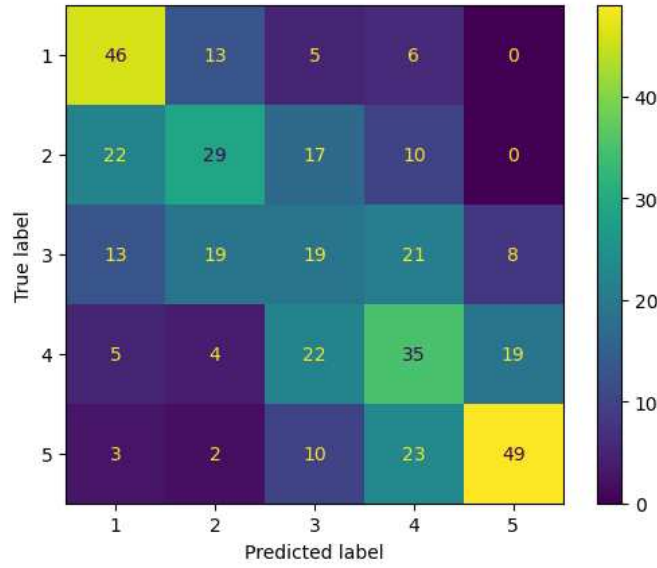


Figure 8.12: Confusion matrix of AutoGluon Tabular classifier trained on abstract + title + LLM features, M17+. Source: author

Feature	Importance	Std. dev.	p-value
abstract	0.1745	0.0233	0.000 037
animal_and_dairy_science	0.0060	0.0022	0.001 941
basic_medicine	0.0060	0.0014	0.000 304
grammar	0.0060	0.0022	0.001 941
earth_and_related_environmental_sciences	0.0045	0.0011	0.000 422
other_medical_sciences	0.0045	0.0021	0.004 290
rigor	0.0035	0.0022	0.012 448
physical_sciences	0.0035	0.0022	0.012 448
health_sciences	0.0035	0.0014	0.002 318
correlational_research	0.0030	0.0011	0.001 941
novelty	0.0020	0.0021	0.049 650
psychology	0.0020	0.0011	0.008 065
veterinary_science	0.0015	0.0014	0.035 242

Table 8.14: Permutation feature importance of AutoGluon Tabular classifier trained on title + abstract + LLM-generated features, M17+. Source: author

8.7 TF-IDF + LLM features

For the training of the following models, the TF-IDF matrix was concatenated to a data frame with the LLM-generated features, resulting in a feature space of 1562 columns - 1500 terms and 62 high-level features.

	Accuracy	Recall	F1 Score	MAE	MSE
Random Forest	0.642	0.643	0.642	0.358	0.358
Gradient Boost	0.650	0.651	0.650	0.350	0.350
AdaBoost	0.645	0.643	0.643	0.355	0.355
Logistic Regression	0.653	0.656	0.653	0.347	0.347
SVM	0.625	0.625	0.625	0.375	0.375
MLP	0.622	0.625	0.621	0.378	0.378

Table 8.15: Performance of best models trained on TF-IDF + LLM features, CORD-19. Green highlights the best model, and red highlights the worst model test accuracy. Source: author

The best model trained on the combined features of the CORD-19 dataset was logistic regression. Fifteen coefficients with the highest and fifteen coefficients with the lowest values in the logistic regression can be seen in Table 8.16. We can see that besides intercept, the term `virus` and features `biological_sciences` and `basic_medicine` are among the most important. It can be seen that none of `rigor`, `accessibility` or `novelty` play an important role.

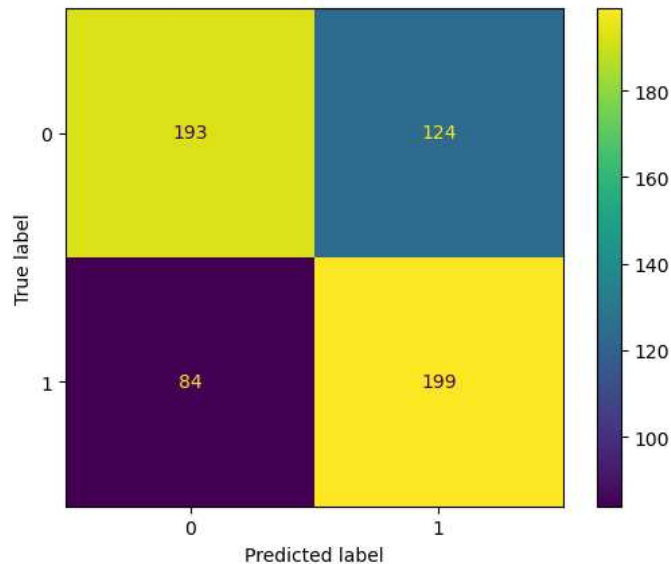


Figure 8.13: Confusion matrix of the best model with TF-IDF + features, CORD-19. Source: author

In the case of ordinal multiclass classification on the M17+ dataset, the models featuring LLM-inferred characteristics and TF-IDF reached very good results, outperforming both

Term/feature (Positive)	Coefficient	Term/feature (Negative)	Coefficient
Intercept	0.0147	study_wa	-0.0075
virus	0.0097	home	-0.0075
biological_sciences	0.0094	index	-0.0071
basic_medicine	0.0092	management	-0.0070
viral	0.0083	education	-0.0066
chemical_sciences	0.0074	experience	-0.0065
agriculture_forestry...	0.0073	chapter	-0.0064
animal_and_dairy_science	0.0070	study	-0.0061
host	0.0069	year_old	-0.0061
infection	0.0068	network	-0.0061
replication	0.0067	provider	-0.0060
china	0.0067	cancer	-0.0060
molecule	0.0066	test	-0.0059
veterinary_science	0.0066	barrier	-0.0059
transmission	0.0066	applied	-0.0058

Table 8.16: Terms and features with highest and lowest regression coefficients, CORD-19.

TF-IDF and LLM-feature-only models. In terms of accuracy, the best model was almost on par with the neural network trained on SciBERT embeddings, although achieving notably worse MAE and MSE.

The Gradient Boost model performed the best in all metrics except for MSE and MAE. AdaBoost achieved much better MSE, but upon inspection, it was shown that while it maintained a low error in terms of MSE, its accuracy was severely lacking.

	Accuracy	Recall	F1 Score	MAE	MSE
Random Forest	0.348	0.343	0.262	1.273	3.162
Gradient Boost	0.393	0.389	0.377	1.068	2.383
Ada Boost	0.313	0.317	0.311	1.028	1.963
Logistic Regression	0.343	0.343	0.307	1.125	2.500
SVM	0.355	0.349	0.339	1.035	2.120
MLP	0.335	0.327	0.303	1.015	1.925

Table 8.17: Performance of best models with TF-IDF + LLM features, M17+. Green highlights the best model, and red highlights the worst model by classification metrics. Source: author

Permutation importance values show us that the key feature in prediction was **rigor**, with value of 0.01925. Other important features were TF-IDF terms, which were terms 'show,' 'aim,' 'could,' and 'data.' We can also see physical sciences and basic medicine as being the

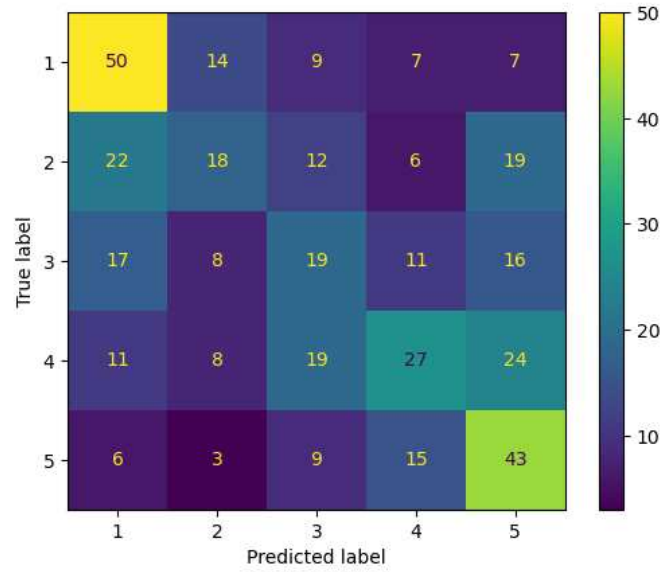


Figure 8.14: Confusion matrix of the Gradient Boost model trained on TF-IDF + features, M17+. Source: author

most impactful features.

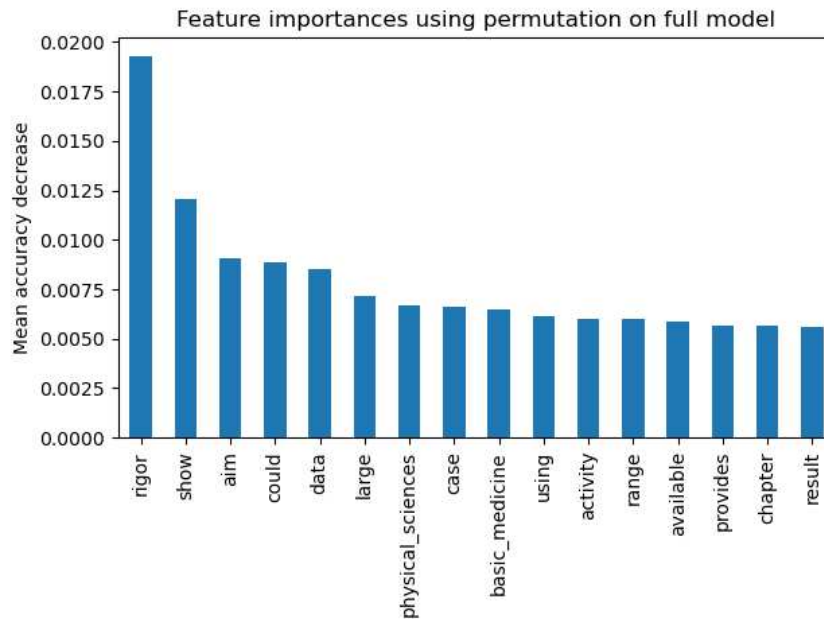


Figure 8.15: Feature importance of the Gradient Boost model trained on TF-IDF + features, M17+. Source: author

8.8 Result comparison

This section compares the predictive performance of models trained on different feature combinations of CORD-19 and M17+ datasets.

For CORD-19, the experiments show that combining text + LLM-generated features trained on the AutoGluon platform yielded the best results and outperformed every dataset variation. It is worth noting that the TF-IDF + LLM-generated features dataset is only 1 percentage point worse regarding accuracy and other metrics. A model with LLM features only performed better than naive classification but failed to match or surpass classification based solely on text data. All models surpassed the dummy classifier by a large margin.

	Accuracy	Recall	F1 Score	MAE	MSE
text + LLM features (AutoGluon)	0.665	0.667	0.664	0.335	0.335
TF-IDF + LLM-features	0.653	0.656	0.653	0.347	0.347
text only (AutoGluon)	0.630	0.636	0.628	0.370	0.370
SciBERT embeddings	0.625	0.625	0.625	0.335	0.375
TF-IDF	0.625	0.622	0.622	0.375	0.375
LLM-features only	0.598	0.598	0.598	0.402	0.402
Dummy classifier	0.502	0.503	0.502	0.499	0.499

Table 8.18: Comparison of best models trained on each dataset variation, CORD-19. Ordered by test accuracy. Source: author

Results for the M17+ dataset show that SciBERT embeddings have the best predictive performance of all metrics besides MSE. Models trained on text + LLM-generated features with AutoGluon almost matched SciBERT, trailing 1 percentage point behind. TF-IDF with LLM-generated features was on par with models trained on aforementioned datasets in terms of classification metrics but overall showed a much larger MAE and MSE. The model with LLM-generated features performed even better than the model trained on the TF-IDF matrix and almost matched the performance of AutoGluon trained only on text. All models surpassed the dummy classifier by a large margin.

	Accuracy	Recall	F1 Score	MAE	MSE
SciBERT embeddings	0.408	0.410	0.392	0.848	1.538
text + LLM features (AutoGluon)	0.395	0.393	0.389	0.855	1.465
TF-IDF + LLM-features	0.393	0.389	0.377	1.068	2.383
text only (AutoGluon)	0.377	0.369	0.324	1.001	2.100
LLM-features only	0.370	0.366	0.338	1.103	2.438
TF-IDF	0.343	0.334	0.332	1.073	2.223
Dummy classifier	0.180	0.180	0.180	1.620	4.020

Table 8.19: Comparison of best models trained on each dataset variation, M17+. Ordered by test accuracy. Source: author

9. Discussion

Application of machine-learning methods on scientific data with the intention of predicting citation count or human evaluation score has been covered by some authors (Beranová et al., 2022; Ngoc, 2024; Thelwall et al., 2023). Still, none have employed features generated from large language models in their experiments.

A large language model was employed and used for generating a total of 7 features - methodological rigor, the novelty of the article, accessibility of language used in the abstract, reproducibility of results, presence of grammar errors, types of research used, and disciplines to which the paper delves deep into.

The predictive performance of generated features was tested in multiple machine-learning experiments with various combinations of LLM-generated features and text data. Furthermore, the permutation importance of generated features in classification tasks was explored.

The generated features did not achieve the predictive results of bibliometric features in other research (Beranová et al., 2022; Ngoc, 2024; Thelwall et al., 2023). However, the main strength of features assessed by large language models is their interpretability. They could provide feedback to article authors when used with certain methods. Action rules (Ras & Wieczorkowska, 2000) could be used to mine associations and show what changes the article authors should undergo to increase their citation count or evaluation score.

Expansion of research is possible by generating more features and using more complex large language models, which should, in theory, provide better results. Adding more features might also help with the explainability of classification and its possible use with action rules. Adjusting the possible values of features, i.e., instead of low, medium, and high, making a scale of 1-10 could also lead to better results. Fine-tuning a large language model on a human-labeled dataset could also increase the performance and impact of features. Working with larger input datasets could potentially increase the predictive performance.

It is necessary to consider whether the gain in predictive performance and interpretability is worth it from an economic standpoint, as in my experiments, extracting features from abstracts costs tens of computing hours, even with relatively small datasets (3000 and 2000 samples).

Conclusion

The main goal of this thesis was to investigate whether features generated by large language models from text data could be used to enhance the predictive performance of machine learning models.

First, two datasets of scientific articles, CORD-19 and M17+, were collected, balanced with respect to the target variables, and preprocessed for further use with LLMs and machine learning algorithms.

Prompts for extraction of features were iteratively constructed and, together with article abstracts, used as inputs for the LLM. For each dataset, a set of 63 categorical features was generated.

These features were then used as inputs for various machine learning models and compared against models based on TF-IDF and SciBERT embeddings. From the results, it is apparent that incorporating these features increases the predictive power of models. All models utilizing LLM-generated features scored above baseline accuracies of dummy classifiers of 50.2% and 18% for CORD-19 and M17+ datasets, respectively.

For CORD-19, models based solely on LLM-generated features did not match the test accuracy of TF-IDF (62.5%) or SciBERT embeddings (62.5%). The best accuracy was shown by models employing a combination of features - AutoGluon with title and abstracts + LLM-generated features (66.5%) and TF-IDF + LLM-generated features model (65.3%).

For M17+, the best model in terms of accuracy was trained on SciBERT embeddings (40.8%). AutoGluon with title, abstract, and LLM-generated features came close in terms of accuracy (39.5%) and even surpassed the SciBERT models in terms of MSE (1.465). The combination of TF-IDF and LLM features showed improvement over the basic TF-IDF approach. It matched the AutoGluon model in classification metrics, falling behind in MAE and MSE.

All models trained on LLM-generated features showed greater predictive power than the baseline dummy classifier. Furthermore, models utilizing both TF-IDF and LLM-generated features consistently improved predictive performance. Features obtained from LLM feature engineering are more interpretable to humans than TF-IDF terms, SciBERT embeddings or bibliometric measures. They could be used with explainable machine learning methods to provide feedback to authors of articles.

The goals of this thesis are considered fulfilled.

Bibliography

- Anil, R., Borgeaud, S., Alayrac, J.-B., Yu, J., Soricut, R., Schalkwyk, J., & Dai, A. M. (2024). Gemini: A family of highly capable multimodal models.
- Annalingam, A., Damayanthi, H., Jayawardena, R., & Ranasinghe, P. (2014). Determinants of the citation rate of medical research publications from a developing country. *SpringerPlus*, 3, 140. <https://doi.org/10.1186/2193-1801-3-140>
- Ante, L. (2021). Readability affects scientific impact: Evidence from emerging technology discourses [Available at SSRN: <https://ssrn.com/abstract=3897998> or <http://dx.doi.org/10.2139/ssrn.3897998>]. *BRL Working Paper Series*, (21).
- Bafna, P., Pramod, D., & Vaidya, A. (2016). Document clustering: Tf-idf approach. *2016 International Conference on Electrical, Electronics, and Optimization Techniques*, 61–66. <https://doi.org/10.1109/ICEEOT.2016.7754750>
- Beel, J., Gipp, B., Langer, S., & Breiteringer, C. (2016). Research-paper recommender systems : A literature survey. *International Journal on Digital Libraries*, 17(4), 305–338. <https://doi.org/10.1007/s00799-015-0156-0>
- Belikov, A. V., & Belikov, V. V. (2015). A citation-based, author- and age-normalized, logarithmic index for evaluation of individual researchers independently of publication counts. *F1000Research*, 4, 884. <https://doi.org/10.12688/f1000research.7070.1>
- Beltagy, I., Lo, K., & Cohan, A. (2019a). Scibert: A pretrained language model for scientific text.
- Beltagy, I., Lo, K., & Cohan, A. (2019b). Scibert: Pretrained language model for scientific text. *EMNLP*.
- Bengio, Y., Ducharme, R., & Vincent, P. (2000). A neural probabilistic language model. In T. Leen, T. Dietterich, & V. Tresp (Eds.), *Advances in neural information processing systems* (Vol. 13). MIT Press. https://proceedings.neurips.cc/paper_files/paper/2000/file/728f206c2a01bf572b5940d7d9a8fa4c-Paper.pdf
- Beranová, L., Joachimiak, M. P., Kliegr, T., Rabby, G., & Sklenák, V. (2022). Why was this cited? explainable machine learning applied to covid-19 research literature. *Scientometrics*, 127(5), 2313–2349. <https://doi.org/10.1007/s11192-022-04314-9>
- Bird, S., Loper, E., & Klein, E. (2009). *Natural language processing with python*. O'Reilly Media Inc.
- Boser, B. E., Guyon, I. M., & Vapnik, V. N. (1992). A training algorithm for optimal margin classifiers. *Proceedings of the Fifth Annual Workshop on Computational Learning Theory*, 144–152. <https://doi.org/10.1145/130385.130401>
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., & Sastry, G. (2020). Language models are few-shot learners. *CoRR*, abs/2005.14165. <https://arxiv.org/abs/2005.14165>

- Bruer, J. T. (1982). Methodological rigor and citation frequency in patient compliance literature. [PMID: 7114334]. *American Journal of Public Health*, 72(10), 1119–1123. <https://doi.org/10.2105/AJPH.72.10.1119>
- Chang, Y., Wang, X., Wang, J., Wu, Y., Yang, L., Zhu, K., Chen, H., Yi, X., Wang, C., Wang, Y., Ye, W., Zhang, Y., Chang, Y., Yu, P. S., Yang, Q., & Xie, X. (2024). A survey on evaluation of large language models [Just Accepted]. *ACM Trans. Intell. Syst. Technol.* <https://doi.org/10.1145/3641289>
- Clark, E., August, T., Serrano, S., Haduong, N., Gururangan, S., & Smith, N. A. (2021, August). All that’s ‘human’ is not gold: Evaluating human evaluation of generated text. In C. Zong, F. Xia, W. Li, & R. Navigli (Eds.), *Proceedings of the 59th annual meeting of the association for computational linguistics and the 11th international joint conference on natural language processing (volume 1: Long papers)* (pp. 7282–7296). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2021.acl-long.565>
- Dietterich, T. G., et al. (2002). Ensemble learning. *The handbook of brain theory and neural networks*, 2(1), 110–125.
- Freund, Y., Schapire, R. E., et al. (1996). Experiments with a new boosting algorithm. *icml*, 96, 148–156.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232. Retrieved June 24, 2024, from <http://www.jstor.org/stable/2699986>
- Gaudette, L., & Japkowicz, N. (2009). Evaluation methods for ordinal classification. In Y. Gao & N. Japkowicz (Eds.), *Advances in artificial intelligence* (pp. 207–210). Springer Berlin Heidelberg.
- Grandini, M., Bagli, E., & Visani, G. (2020). Metrics for multi-class classification: An overview.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Hosmer Jr, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied logistic regression*. John Wiley & Sons.
- Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Jia, Y., Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, ... Xiaoqiang Zheng. (2015). TensorFlow: Large-scale machine learning on heterogeneous systems [Software available from tensorflow.org]. <https://www.tensorflow.org/>
- McHugh, M. L. (2013). The chi-square test of independence. *Biochemia Medica*, 23(2), 143–149. <https://doi.org/10.11613/bm.2013.018>
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space.
- Molnar, C. (2022). *Interpretable machine learning: A guide for making black box models explainable* (2nd ed.). <https://christophm.github.io/interpretable-ml-book>

- Nagel, M., Fournarakis, M., Amjad, R. A., Bondarenko, Y., van Baalen, M., & Blankevoort, T. (2021). A white paper on neural network quantization.
- Ngoc, B. C. (2024). Predikce hodnocení vědeckých prací [online] [SUPERVISOR: Tomáš Kliegr]. <https://theses.cz/id/pd9t84/>
- OECD. (2015). *Frascati manual 2015*. <https://doi.org/https://doi.org/https://doi.org/10.1787/9789264239012-en>
- Okamura, K. (2019). Interdisciplinarity revisited: Evidence for research impact and dynamism. *Palgrave Communications*, 5(1), 141. <https://doi.org/10.1057/s41599-019-0352-4>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Powers, D. (2008). Evaluation: From precision, recall and f-factor to roc, informedness, markedness correlation. *Mach. Learn. Technol.*, 2.
- Rada pro výzkum, vývoj a inovace. (2017, February). *Hodnocení výzkumných organizací a hodnocení programů účelové podpory výzkumu, vývoje a inovací dle metodiky m17+* [Accessed: 2024-06-23]. Výzkum a vývoj v ČR. <https://vyzkum.gov.cz/FrontClanek.aspx?idsekce=799796>
- Raff, E. (2022). Does the market of citations reward reproducible work? <https://arxiv.org/abs/2204.03829>
- Rajaraman, A., & Ullman, J. D. (2011). Data mining. In *Mining of massive datasets* (pp. 1–17). Cambridge University Press.
- Ras, Z. W., & Wiczkorkowska, A. (2000). Action-rules: How to increase profit of a company. In D. A. Zighed, J. Komorowski, & J. Żytkow (Eds.), *Principles of data mining and knowledge discovery* (pp. 587–592). Springer Berlin Heidelberg.
- Reimers, N., & Gurevych, I. (2019). Sentence-bert: Sentence embeddings using siamese bert-networks. *CoRR*, abs/1908.10084. <http://arxiv.org/abs/1908.10084>
- Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with python. *9th Python in Science Conference*.
- Serra-Garcia, M., & Gneezy, U. (2021). Nonreplicable publications are cited more than replicable ones [Distributed under a Creative Commons Attribution NonCommercial License 4.0 (CC BY-NC)]. *Science advances*, 7(21), eabd1705. <https://doi.org/10.1126/sciadv.abd1705>
- Sparck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1), 11–21. <https://doi.org/10.1108/eb026526>
- Tang, Z., Fang, H., Zhou, S., Yang, T., Zhong, Z., Hu, T., Kirchhoff, K., & Karypis, G. (2024). Autogluon-multimodal (automm): Supercharging multimodal automl with foundation models. *arXiv preprint arXiv:2404.16233*.
- Thelwall, M., Kousha, K., Wilson, P., Makita, M., Abdoli, M., Stuart, E., Levitt, J., Knoth, P., & Cancellieri, M. (2023). Predicting article quality scores with machine learning: The U.K. Research Excellence Framework. *Quantitative Science Studies*, 4(2), 547–573. https://doi.org/10.1162/qss_a_00258

- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Ferrer, C. C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., . . . Scialom, T. (2023). Llama 2: Open foundation and fine-tuned chat models.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2023). Attention is all you need.
- Wang, J., Veugelers, R., & Stephan, P. (2017). Bias against novelty in science: A cautionary tale for users of bibliometric indicators. *Research Policy*, 46(8), 1416–1436. <https://doi.org/https://doi.org/10.1016/j.respol.2017.06.006>
- Wang, L. L., Lo, K., Chandrasekhar, Y., Reas, R., Yang, J., Eide, D., Funk, K., Kinney, R. M., Liu, Z., Merrill, W., Mooney, P., Murdick, D. A., Rishi, D., Sheehan, J., Shen, Z., Stilson, B., Wade, A. D., Wang, K., Wilhelm, C., . . . Kohlmeier, S. (2020). Cord-19: The covid-19 open research dataset. *ArXiv*. <https://api.semanticscholar.org/CorpusID:216056360>
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., & Brew, J. (2019). Huggingface’s transformers: State-of-the-art natural language processing. *CoRR*, abs/1910.03771. <http://arxiv.org/abs/1910.03771>

Appendices

A. Prompts used for feature extraction.

```
1 '''[INST] <<SYS>>
2 You are a categorization assistant. Your job will be to assign a certain
3 characteristic to a research paper based on its abstract.
4 In this instance you will decide the novelty of the research and its
5 findings based on the abstract.
6 Novelty is a metric of how innovative the research presented is and how
7 much it is new to the landscape of scientific research.
8 You will choose between three levels of novelty: low, medium and high.
9 Low being the least innovative and high being the most.
10 Be concise, do not provide any explanations. Your answer will consist of
11 an answer in JSON format like so:
12 {{
13     "novelty": "value"
14 }}
15 <</>>
16 Abstract to be evaluated:
17 {prompt}[/INST]
18 Response:"
```

Listing A.1: Prompt used for extraction of the novelty feature. Source: author

```
1 '''[INST] <<SYS>>
2 You are a categorization assistant. Your job will be to assign a certain
3 characteristic to a research paper based on its abstract.
4 In this instance you will assess the accessibility of the research.
5 Accessibility is a metric of readability and understandability of the
6 language used. Abstracts that do not use overly complicated language while
7 easily explaining what the paper is about will be marked as accessible.
8 You will choose between three levels of accessibility: low, medium and
9 high. Low being the least understandable and high being the most.
10 Be concise, do not provide any explanations. Your answer will consist of
11 an answer in json format like so:
12 {{
13     "accessibility": "value"
14 }}
15 <</>>
16 Abstract to be evaluated:
17 {prompt}[/INST]
18 Response:
19 '''
```

Listing A.2: Prompt used for extraction of the accessibility feature. Source: author

```
1 '''[INST] <<SYS>>
2 You are a categorization assistant. Your job will be to assign a certain
3 characteristic to a research paper based on its abstract.
4 In this instance you will assess the methodological rigor of the research
5 .
```

```

4   Methodological rigor represents the soundness of logic presented in the
   article abstract by its authors.
5   You will choose between three levels of rigor: low, medium and high. Low
   being the least rigorous and high being the most.
6   Be concise, do not provide any explanations. Your answer will consist of
   an answer in json format like so:
7   {{
8       "rigor": "value"
9   }}
10  <</>>
11  Abstract to be evaluated:
12  {prompt}[/INST]
13  Response:
14  '''

```

Listing A.3: Prompt used for extraction of the rigor feature. Source: author

```

1  '''[INST] <<SYS>>
2   You are a categorization assistant. Your job will be to assign a certain
   characteristic to a research paper based on its abstract.
3   In this instance, you will decide whether the abstract concerns the
   scientific discipline of {ford_categories[discipline]}.
4   This discipline consists of these subcategories: {get_subclasses(
   discipline)} - the paper only qualifies as a part of this discipline if it
   directly touches any of these topics as one of the main goals of the
   paper.
5   You will choose between two values in your reply: 0 or 1. 0 means that
   this research paper does not in any significant way delve deep into this
   field.
6   1 means that this paper and its research is closely tied to this field.
7   Note that in order to classify as a part of {ford_categories[discipline]}
   discipline, it has to delve deeper than only use certain methods from
   that discipline.
8   Be concise, do not provide any explanations. Your answer will consist
   only of the number and nothing else. Be as accurate as possible.
9   <</>>
10
11  Title: {title}
12  Abstract:
13  {abstract}[/INST]
14  Response:
15  '''

```

Listing A.4: Prompt used for extraction of the rigor feature. Source: author

```

1  '''[INST] <<SYS>>
2   You are a categorization assistant. Your job will be to assign a certain
   characteristic to a research paper based on its abstract.
3   In this instance you will decide the rigor of the logic presented in the
   abstract.
4   Your job will be to assess the methodological soundness and the quality
   of logic provided in the abstract.
5   You will choose between three levels of rigor: low, medium and high. Low
   being the least rigorous and high being the most.

```

```
6   Be concise, do not provide any explanations. Your answer will consist of
   an answer in json format like so:
7   {{
8       "rigor": "value"
9   }}
10  <</>>
11  Abstract to be evaluated:
12  {abstract}[/INST]
13  Response:
14  '''
```

Listing A.5: Prompt used for extraction of the rigor feature. Source: author

```
1  '''[INST] <<SYS>>
2    You are a categorization assistant. Your job will be to assign a certain
   characteristic to a research paper based on its abstract.
3    In this instance you will assess whether the abstract has grammatical
   errors in it. All of the abstracts should be written in english - British
   or American.
4    You will choose between two values in your reply: 0 or 1. 0 being error-
   free abstract, 1 being grammatically incorrect one.
5    Be concise, do not provide any explanations. Your answer will consist of
   an answer in json format like so:
6    {{
7        "grammar": "value"
8    }}
9    <</>>
10   Abstract to be evaluated:
11   {prompt}[/INST]
12   Response:
13   '''
```

Listing A.6: Prompt used for extraction of the grammar feature. Source: author

```
1  '''[INST] <<SYS>>
2    You are a categorization assistant. Your job will be to assign a certain
   characteristic to a research paper based on its abstract.
3    In this instance you will assess whether the abstract has grammatical
   errors in it. All of the abstracts should be written in english - British
   or American.
4    You will choose between two values in your reply: 0 or 1. 0 being error-
   free abstract, 1 being grammatically incorrect one.
5    Be concise, do not provide any explanations. Your answer will consist of
   an answer in json format like so:
6    {{
7        "grammar": "value"
8    }}
9    <</>>
10   Abstract to be evaluated:
11   {prompt}[/INST]
12   Response:
13   '''
```

Listing A.7: Prompt used for extraction of the grammar feature. Source: author

```

1 '''[INST] <<SYS>>
2     You are a categorization assistant. Your job will be to assign a certain
3     characteristic to a research paper based on its abstract.
4     In this instance you will assess whether the abstract mentions
5     reproducibility of results..
6     You will choose between two values in your reply: 0 or 1. 0 meaning zero
7     mentions of reproducibility, 1 meaning authors have addressed
8     reproducibility of their research .
9     Be concise, do not provide any explanations. Your answer will consist of
10    an answer in json format like so:
11    {{
12        "replicability": "value"
13    }}
14    <</>>
15    Abstract to be evaluated:
16    {prompt}[/INST]
17    Response:
18    '''

```

Listing A.8: Prompt used for extraction of the replicability feature. Source: author

B. Feature generation

File `generate_features.ipynb` shows in detail the code used to generate features based on abstracts.

C. Stastistical tests of all features

File `feature_analysis.ipynb` shows the process and results of the statistical chi-squared test of independence on all generated features for both datasets.

D. Train-test split

Files `test_c19.csv`, `train_c19.csv`, `test_m17.csv` and `train_m17.csv` contain train-test splits used for experiments with different feature subsets.